ACTIVIDAD SUMATIVA: UNIDAD 4
CREACIÓN DE APLICACIÓN USANDO EL PATRÓN SERVERLESS
CONTENERIZACIÓN DE APLICACIONES DE SOFTWARE

Pokédex Serverless: Cloudflare Workers y D1

PRESENTADO POR:
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
ING. VÍCTOR AUGUSTO PEDRAZA BONILLA

POLITÉCNICO GRANCOLOMBIANO
MAESTRÍA EN ARQUITECTURA DE SOFTWARE
BOGOTÁ D.C.
7 DE NOVIEMBRE
2026

TABLA DE CONTENIDO

1. INTRODUCCIÓN 3

2. DATASET SELECCIONADO 5

2.1 Fuente y alcance 5

2.2 Estructura de columnas 5

2.3 Hallazgos de calidad de datos 5

3. ARQUITECTURA SERVERLESS Y JUSTIFICACIÓN DE COMPONENTES 6

3.1 Componentes y justificación 6

3.2 Patrones aplicados 7

3.3 Por qué Cloudflare y no otro proveedor 8

3.4 Por qué serverless y no contenedores en esta actividad 8

4. BASE DE DATOS SERVERLESS: MODELO, SCRIPT Y EVIDENCIA 8

4.1 Modelo de datos 8

4.2 Consultas y su costo de acceso 9

4.3 Script de creación y carga 9

4.4 Verificación y evidencia 9

5. APLICACIÓN WEB SERVERLESS 10

5.1 Backend: un Worker con Chain of Responsibility 10

5.2 Frontend: JavaScript puro, sin dependencias 11

6. DESPLIEGUE EN CLOUDFLARE 13

6.1 Requisitos y pasos 13

6.2 Comprobación de la API publicada 14

6.3 Dificultades encontradas 14

6.4 Evidencia del despliegue 14

7. PRUEBAS Y RESULTADOS 16

8. VIDEO EXPLICATIVO 17

9. CONCLUSIONES 18

10. BIBLIOGRAFÍA 18

11. ANEXO A: ENTREGA DEL CÓDIGO Y EJECUCIÓN 19

12. ANEXO B: CÓDIGO FUENTE 20

B.1 wrangler.toml 20

B.2 db/schema.sql 20

B.3 scripts/build_seed.py 21

B.4 src/worker.js 23

B.5 src/chain.js 24

B.6 src/repository.js 26

B.7 frontend/index.html 29

B.8 frontend/styles.css 30

B.9 frontend/app.js 34

13. AGRADECIMIENTO 40

FICHA DEL DOCUMENTO

Proyecto	Pokédex Serverless: aplicación web sobre Cloudflare Workers y D1
Documento	Actividad sumativa, Unidad 4: Creación de aplicación usando el patrón serverless
Módulo	Contenerización de Aplicaciones de Software
Programa	Maestría en Arquitectura de Software, Politécnico Grancolombiano
Versión	1.0 (noviembre de 2026)
Fecha	Noviembre de 2026
Autor	Alejandro De Mendoza
Docente	Ing. Víctor Augusto Pedraza Bonilla
Repositorio	https://github.com/AlejoTechEngineer/pokedex-serverless
Aplicación publicada	https://pokedex-serverless.pokedex-serverless.workers.dev
Video explicativo	https://youtu.be/TodbzdBe8oU
Duración del video	33 minutos y 13 segundos
Calidad de reproducción	Ver en 1080p HD: engranaje de YouTube, «Calidad», «1080p»
Servicios utilizados	Cloudflare Workers, Cloudflare D1 y Workers Static Assets
Tecnologías	JavaScript, SQL (SQLite), Python, wrangler, HTML y CSS

1. INTRODUCCIÓN

La evolución de las arquitecturas de software y de los modelos de computación en la nube ha impulsado la adopción de enfoques orientados a desarrollar, desplegar y operar aplicaciones con mayor flexibilidad, escalabilidad y eficiencia. En este contexto, la computación *serverless* representa una alternativa relevante frente a los modelos tradicionales de infraestructura, al permitir que los desarrolladores concentren sus esfuerzos en la lógica de negocio y en las funcionalidades de las aplicaciones, mientras que el aprovisionamiento, la administración y buena parte de la operación de los recursos computacionales son gestionados por el proveedor de servicios en la nube. Este enfoque adquiere especial importancia en escenarios donde las cargas de trabajo son variables, las operaciones tienen una duración limitada y se requiere reducir la complejidad asociada con la administración de servidores.

El presente trabajo se desarrolla en el marco de la **Unidad 4 del módulo Contenerización de Aplicaciones de Software**, perteneciente a la Maestría en Arquitectura de Software del Politécnico Grancolombiano. Su propósito es aplicar los conceptos relacionados con los patrones de diseño y despliegue de aplicaciones *serverless* mediante la construcción de una solución funcional en un entorno cloud. Como antecedente, la Unidad 3 abordó la orquestación de contenedores mediante Kubernetes, permitiendo comprender los mecanismos de despliegue, administración y escalabilidad de aplicaciones basadas en contenedores. A partir de estos conocimientos, la Unidad 4 amplía la perspectiva hacia modelos de ejecución administrados por proveedores cloud, incluyendo el despliegue de aplicaciones y microservicios mediante funciones *serverless*.

Para materializar estos conceptos, se desarrolla **Pokédex Serverless**, una aplicación web que permite consultar, visualizar y analizar información de un conjunto de datos público relacionado con Pokémon. La solución utiliza un dataset de **1.215 registros**, que se procesa y almacena en **Cloudflare D1**, una base de datos SQL administrada dentro del ecosistema de Cloudflare. La información se consulta mediante un **Worker de Cloudflare**, encargado de recibir las solicitudes del cliente, ejecutar la lógica correspondiente y devolver los resultados a través de una API. De esta manera, se establece

una separación funcional entre la persistencia de los datos, el procesamiento de las solicitudes y la presentación de la información en la interfaz web, aprovechando los servicios administrados de la plataforma.

La aplicación incorpora funcionalidades de consulta por identificador y nombre, filtros por categorías y generación, visualización tabular de registros, presentación de fichas individuales y representación gráfica de características estadísticas. Estas funcionalidades permiten demostrar la interacción entre los componentes de la solución y evidenciar cómo una aplicación basada en servicios *serverless* puede ofrecer capacidades de consulta y visualización sin requerir la administración directa de una infraestructura de servidores. Asimismo, el proyecto permite abordar aspectos propios de la ingeniería de software, como el modelado de datos, la selección de componentes tecnológicos, la organización de la lógica de procesamiento, la validación de solicitudes y la comprobación del funcionamiento mediante pruebas.

Desde la perspectiva arquitectónica, el trabajo aplica el patrón de despliegue de **microservicios serverless**, abordado en la lectura fundamental de la Unidad 4, sección 5, como fundamento para comprender la ejecución de componentes mediante servicios administrados. Adicionalmente, se incorpora el patrón **Chain of Responsibility**, descrito en la sección 2 de la misma lectura, para organizar el enrutamiento y procesamiento de las solicitudes dentro del Worker. La combinación de estos enfoques permite relacionar los conceptos estudiados con una implementación práctica, diferenciando las decisiones asociadas al despliegue de la aplicación de aquellas que corresponden a la estructuración interna de su lógica.

El desarrollo contempla, además, la selección y caracterización del dataset, el diseño del esquema de base de datos, la construcción de los scripts de creación y carga, la implementación de los componentes de backend y frontend, y el despliegue de la aplicación en Cloudflare. Finalmente, se presentan las pruebas realizadas y las evidencias obtenidas durante la implementación, con el propósito de documentar el proceso de desarrollo y verificar el comportamiento de la solución frente a los requerimientos establecidos en la actividad.

El documento se organiza de acuerdo con la secuencia del enunciado. La **sección 2** presenta el dataset seleccionado y sus características; la **sección 3** describe la arquitectura propuesta y justifica la elección de sus componentes tecnológicos; la **sección 4** desarrolla el modelo de datos, los scripts de creación y carga, y las evidencias de la base de datos; la **sección 5** expone la implementación de la aplicación web; la **sección 6** documenta el despliegue en Cloudflare; la **sección 7** presenta las pruebas y los resultados obtenidos; y la **sección 8** describe el video explicativo de la solución. El código fuente completo se encuentra disponible en el repositorio https://github.com/AlejoTechEngineer/pokedex-serverless y se complementa con el **Anexo B**, en el que se incorpora el código correspondiente a los principales componentes del proyecto.

A continuación, se ilustra el cumplimiento de las condiciones de entrega establecidas para la actividad y su correspondencia con las secciones y evidencias incluidas en el presente documento.

Tabla 1. Condiciones de entrega y dónde se cumplen.

Condición de entrega	Dónde está
Código fuente de la aplicación web	Sección 5 y Anexo B (Worker y frontend)
Script de la base de datos	Sección 4 y Anexo B (`schema.sql`, `build_seed.py`, `wrangler.toml`)
Enlace del video, público	Sección 8
Aplicación publicada	https://pokedex-serverless.pokedex-serverless.workers.dev

2. DATASET SELECCIONADO

El conjunto de datos seleccionado constituye el punto de partida para el desarrollo de Pokédex Serverless, ya que proporciona la información que será almacenada, procesada y consultada a través de los servicios implementados en la nube. En esta sección se presenta la fuente de información utilizada, su estructura y las características de los registros que lo componen, identificando las variables disponibles y su utilidad dentro de las funcionalidades de la aplicación. Adicionalmente, se examinan las particularidades y condiciones de calidad encontradas durante su revisión, con especial atención a la consistencia de los identificadores, los valores vacíos y la integridad de los registros. Este análisis permite establecer las consideraciones necesarias para preparar los datos antes de su incorporación a Cloudflare D1 y garantizar su correcta utilización en los procesos de consulta y visualización de la aplicación.

2.1 Fuente y alcance

Se usó una lista pública de Pokémon en formato CSV (repositorio `lgreski/pokemonData` en GitHub, archivo `Pokemon.csv`), alternativa permitida por la actividad, que admite un conjunto de datos de Kaggle o de otra fuente de preferencia. Tiene **1.215 filas y 13 columnas**, y 1.025 valores distintos de ID. Es un dominio con claves, categorías y seis métricas numéricas, adecuado para mostrar los datos en tabla, tarjeta y gráficas.

2.2 Estructura de columnas

La Tabla 2 relaciona cada columna original con su columna en la base de datos. Los nombres se normalizan a minúsculas con guion bajo, y el tipo de cada columna se infiere observando todos sus valores.

Tabla 2. Columnas del dataset y su correspondencia en Cloudflare D1.

Columna original	Columna en D1	Tipo	Nota
ID	`id`	INTEGER	Parte de la clave primaria. No es único en el CSV
Form	`form`	TEXT	Parte de la clave primaria. «Base» cuando viene vacío (985 casos)
Name	`name`	TEXT	Se añade `name_lower` para buscar sin distinguir mayúsculas
Type1	`type1`	TEXT	Indexada junto con `id`
Type2	`type2`	TEXT	`NULL` cuando está vacío (546 casos)
Total, HP, Attack, Defense, Sp. Atk, Sp. Def, Speed	`total`, `hp`, `attack`, `defense`, `sp_atk`, `sp_def`, `speed`	INTEGER	Siete métricas numéricas
Generation	`generation`	INTEGER	Indexada junto con `id`

2.3 Hallazgos de calidad de datos

Antes de realizar la carga del conjunto de datos en la base de datos, se llevó a cabo una revisión de su estructura y consistencia con el propósito de identificar posibles irregularidades que pudieran afectar el almacenamiento y la posterior consulta de la información. Este análisis permitió reconocer particularidades relacionadas con la unicidad de los identificadores, la presencia de valores vacíos y posibles colisiones entre registros. A partir de estos hallazgos, se establecieron mecanismos de normalización y tratamiento durante el proceso de generación del script de carga, procurando conservar la totalidad de los registros originales y garantizar la integridad de la información. A

continuación, se presentan los principales hallazgos identificados y las medidas aplicadas para resolverlos.

- **El ID no es único.** 153 especies repiten ID por sus formas alternas (Mega, Alola, Galar). La clave primaria es compuesta: `id` + `form`.
- **Celdas con un espacio en lugar de vacío** en `Form` y `Type2`. El generador aplica `strip()`: `Type2` vacío pasa a `NULL` y `Form` vacío pasa a «Base», porque una clave no puede ser nula.
- **Colisión real de clave.** `Zygarde50% Forme` y `Zygarde10% Forme` llegan con `Form` vacío y el mismo ID 718. En lugar de descartar filas, el generador las desambigua usando el nombre y lo reporta.
- **Sin pérdida de datos.** Al final se verifica que el conteo en la base sea igual al del CSV: 1.215 = 1.215.

3. ARQUITECTURA SERVERLESS Y JUSTIFICACIÓN DE COMPONENTES

Cada petición HTTP a `/api/*` activa un Worker de Cloudflare; el Worker consulta la base D1 y responde en JSON. Los archivos del sitio los sirve Cloudflare directamente. No existe ningún servidor, contenedor ni clúster que el equipo deba aprovisionar, parchear o escalar. La Figura 1 resume los dos caminos que sigue el navegador.

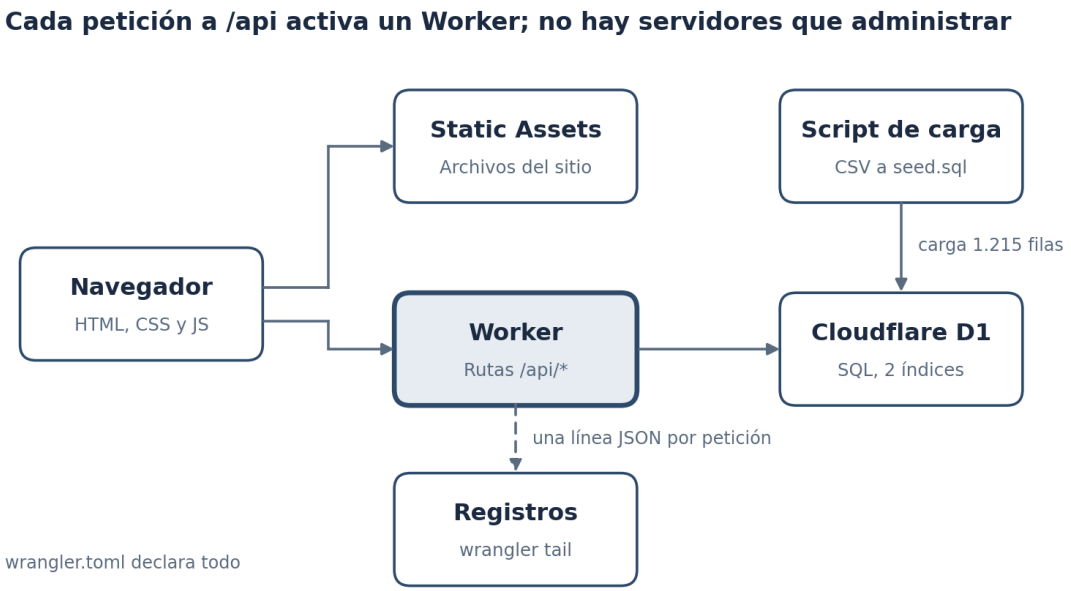


Figura 1. Arquitectura serverless en Cloudflare: el sitio estático y la API del Worker sobre D1.

3.1 Componentes y justificación

La arquitectura de Pokédex Serverless se estructura a partir de un conjunto de componentes tecnológicos que trabajan de manera integrada para garantizar el almacenamiento, procesamiento, consulta y visualización de la información en un entorno completamente administrado en la nube. La selección de cada servicio responde a criterios de funcionalidad, simplicidad operativa, eficiencia y coherencia con los principios del modelo serverless. Asimismo, se consideraron aspectos como la comunicación entre componentes, la configuración de los recursos, la seguridad en el acceso a los datos y la observabilidad de las operaciones. A continuación, en la tabla 3 se describen los

componentes que conforman la solución, los servicios utilizados, las razones que justifican su elección y las características principales de su configuración.

Tabla 3. Componentes de la arquitectura y su justificación.

Componente	Servicio	Por qué se eligió	Estructura y configuración
Base de datos	Cloudflare D1 (SQL sobre SQLite)	Base serverless administrada: sin servidor de base de datos que operar y con SQL estándar. El plan gratuito no pide tarjeta y, al superar el límite diario, las consultas fallan en lugar de facturarse	Tabla `pokemon` con clave `(id, form)`, 2 índices secundarios, sin `ROWID`
Cómputo	Cloudflare Workers	Funciones que se ejecutan por petición en aislados V8, sin servidor propio y con escalado automático	Un Worker en módulos ES: `worker.js`, `chain.js`, `repository.js`. Acceso a la base solo por el enlace `DB`, sin credenciales en el código
API	Rutas `/api/*` del mismo Worker	Sin gateway aparte: HTTPS automático y mismo origen que el sitio, por lo que no hace falta CORS	`GET /api/health`, `/api/pokemon/{id}`, `/api/pokemon`, `/api/stats`; solo lectura
Frontend	Workers Static Assets	Contenido estático servido por Cloudflare sin ejecutar código	Carpeta `frontend/`; con `run_worker_first` solo `/api/*` ejecuta el Worker
Configuración	`wrangler.toml` y `wrangler`	Todo se declara en un archivo y se despliega con un comando	Worker, activos estáticos, enlace a D1 y observabilidad activada
Observabilidad	Workers Logs y `wrangler tail`	Una línea JSON por petición (ruta, estado, duración)	Registro estructurado desde `worker.js`
Carga de datos	`build_seed.py` y `wrangler d1 execute`	Convierte el CSV en SQL idempotente y lo carga igual en local y en la nube	`db/seed.sql`: 1.215 sentencias `INSERT OR REPLACE`

3.2 Patrones aplicados

La implementación de la solución se fundamenta en la aplicación de dos patrones que cumplen funciones complementarias dentro de la arquitectura: el despliegue de microservicios serverless, que define el modelo de ejecución y despliegue del componente de procesamiento en la nube, y Chain of Responsibility, que organiza internamente la gestión y el enrutamiento de las solicitudes HTTP. La combinación de ambos permite abordar tanto la forma en que se ejecuta la aplicación como la manera en que se estructura su lógica, favoreciendo la modularidad, la separación de responsabilidades y la facilidad de evolución del software. A continuación, se explica cómo se aplican estos patrones en el desarrollo de Pokédex Serverless y su relación con los principios de diseño estudiados en la Unidad 4.

Despliegue de microservicios serverless (Unidad 4, sección 5). La lectura lo describe como funciones desplegadas de forma independiente, activadas por eventos, con escalado automático y pago por ejecución. Aquí el Worker es ese microservicio: cada petición HTTP es el evento que lo activa, se despliega con un comando y no consume recursos mientras nadie lo consulta.

Chain of Responsibility (sección 2). Dentro del Worker, la ruta se resuelve con una cadena de manejadores independientes: `MethodGuard → Health → PokemonById → PokemonList → Stats → 404`. Cada uno decide si atiende la petición o la delega al siguiente. Agregar un endpoint es agregar una clase, sin tocar las existentes (principio abierto/cerrado), y cada clase tiene una sola responsabilidad.

3.3 Por qué Cloudflare y no otro proveedor

La actividad pide una base de datos serverless y funciones serverless, y cita servicios de AWS, Google y Azure solo como ejemplos. Cloudflare D1 y Workers cumplen esa misma definición, y se eligieron por tres razones: el plan gratuito no exige método de pago, no vence con el tiempo, y no genera cobros inesperados porque al agotar el límite diario las consultas se rechazan. Se descartó una cuenta de AWS con el periodo gratuito ya vencido, donde cualquier recurso olvidado se factura.

Límites aceptados: los topes diarios de lectura y escritura del plan gratuito, y que la búsqueda parcial por nombre recorre la tabla, algo viable con unos 1.200 registros.

3.4 Por qué serverless y no contenedores en esta actividad

En la Unidad 3 se orquestaron contenedores con Kubernetes, que ofrece control total pero exige administrar el clúster, los nodos y el escalado. Esta aplicación tiene carga intermitente y operaciones cortas de lectura, que es exactamente el caso donde la lectura recomienda serverless; para cargas constantes o tareas largas sería mejor otra arquitectura.

4. BASE DE DATOS SERVERLESS: MODELO, SCRIPT Y EVIDENCIA

La base de datos constituye uno de los componentes fundamentales de la arquitectura de Pokédex Serverless, ya que permite almacenar y organizar la información del conjunto de datos seleccionado para su posterior consulta desde la aplicación web. En esta sección se presenta el diseño del modelo relacional implementado mediante Cloudflare D1, considerando las particularidades identificadas durante el análisis de calidad de los datos y los requerimientos funcionales de la solución. Asimismo, se describen la estructura de las tablas, las claves e índices utilizados, los mecanismos de generación y ejecución del script de carga y los procedimientos de verificación aplicados. Finalmente, se incluyen las evidencias que permiten comprobar la creación de la base de datos, la incorporación de los registros y la correspondencia entre la información original y los datos almacenados en el entorno cloud.

4.1 Modelo de datos

La base `pokedex` de **Cloudflare D1** es una base SQL administrada: no hay servidor de base de datos que instalar ni dimensionar. La tabla `pokemon` usa la clave primaria compuesta `(id, form)` y se almacena sin `ROWID`, ordenada por esa clave. Dos índices secundarios resuelven los filtros por tipo y por generación, ya ordenados por `id`.

```
CREATE TABLE IF NOT EXISTS pokemon (
  id          INTEGER NOT NULL,
  form        TEXT    NOT NULL,            -- "Base" cuando el CSV viene vacío
  name        TEXT    NOT NULL,
  name_lower  TEXT    NOT NULL,            -- para búsqueda sin distinguir mayúsculas
  type1       TEXT    NOT NULL,
  type2       TEXT,                        -- NULL en los de un solo tipo
  total       INTEGER NOT NULL,
  hp          INTEGER NOT NULL,
  attack      INTEGER NOT NULL,
  defense     INTEGER NOT NULL,
  sp_atk      INTEGER NOT NULL,
  sp_def      INTEGER NOT NULL,
  speed       INTEGER NOT NULL,
  generation  INTEGER NOT NULL,
  PRIMARY KEY (id, form)
) WITHOUT ROWID;
```

```
-- Índices secundarios: filtrar por tipo o generación ya ordenado por id.
CREATE INDEX IF NOT EXISTS idx_pokemon_type       ON pokemon (type1, id);
CREATE INDEX IF NOT EXISTS idx_pokemon_generation ON pokemon (generation, id);
```

4.2 Consultas y su costo de acceso

Cada consulta que hace la aplicación se resuelve con la ruta de acceso más económica que ofrece el modelo de datos. La Tabla 4 relaciona cada consulta con la forma en que la base la resuelve (la clave primaria, uno de los dos índices secundarios o un recorrido de la tabla) y con el costo de acceso que implica, es decir, cuántas filas debe leer D1 para responderla. Con ello se ve por qué la búsqueda por ID y los filtros leen solo las filas necesarias, y por qué la búsqueda parcial por nombre, que sí recorre la tabla, es aceptable con unos 1.200 registros.

Tabla 4. Cómo se resuelve cada consulta de la aplicación.

Consulta de la aplicación	Cómo se resuelve	Costo de acceso
Buscar por ID	Clave primaria (`id, form`)	Lee solo las formas de esa especie
Filtrar por tipo	Índice `idx_pokemon_type`	Lee solo ese tipo, ya ordenado por `id`
Filtrar por generación	Índice `idx_pokemon_generation`	Lee solo esa generación, ya ordenada por `id`
Paginar	Cursor por clave (`id, form`), sin `OFFSET`	Cada página cuesta lo mismo, sin importar la profundidad
Buscar por nombre (parcial)	`LIKE` sobre `name_lower`, con comodines escapados	Recorre la tabla; aceptable con ~1.200 filas
Estadísticas del panorama	4 consultas agregadas en un solo lote	Guardadas 60 s en el aislado del Worker

4.3 Script de creación y carga

El script `scripts/build_seed.py` (Anexo B) lee el CSV, normaliza los nombres a minúsculas con guion bajo, infiere el tipo de cada columna observando todos sus valores, convierte las celdas vacías en `NULL` (y `Form` vacío en «Base»), resuelve las colisiones de clave y genera `db/seed.sql` con 1.215 sentencias `INSERT OR REPLACE`. Es idempotente: cargarlo dos veces no duplica registros.

```
python scripts/build_seed.py
npx wrangler d1 execute pokedex --remote --file db/schema.sql
npx wrangler d1 execute pokedex --remote --file db/seed.sql
npx wrangler d1 execute pokedex --remote --command "SELECT COUNT(*) AS filas, COUNT(DISTINCT id) AS
ids FROM pokemon"
```

Salida del generador:

```
· CSV leído: 1215 filas · 1025 valores distintos de id
· Colisiones (id, form) resueltas con el nombre: ['718:Zygarde50% Forme', '718:Zygarde10% Forme']
· Celdas vacías: form=985 (-> 'Base'), type2=546 (-> NULL)
· Escrito db/seed.sql con 1215 filas en 25 lotes
```

4.4 Verificación y evidencia

El resultado de la verificación, tanto en la base local como en la base en la nube de Cloudflare, es **1.215 filas y 1.025 IDs distintos**, igual que el CSV. La Figura 2 muestra la base `pokedex` en el panel de Cloudflare, con una tabla y 152 kB de almacenamiento. La Figura 3 muestra la estructura creada en la base remota: la tabla pokemon y sus dos índices, idx_pokemon_type e idx_pokemon_generation.

La Figura 4 muestra las consultas SQL ejecutadas sobre ella y las filas leídas por cada una; entre ellas aparece la verificación `SELECT COUNT(*) AS filas, COUNT(DISTINCT id) AS ids`, que recorrió unas 1,22 mil filas.

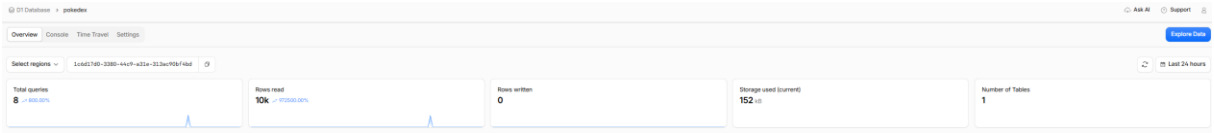


Figura 2. Base de datos D1 pokedex en el panel de Cloudflare: identificador, almacenamiento y número de tablas.

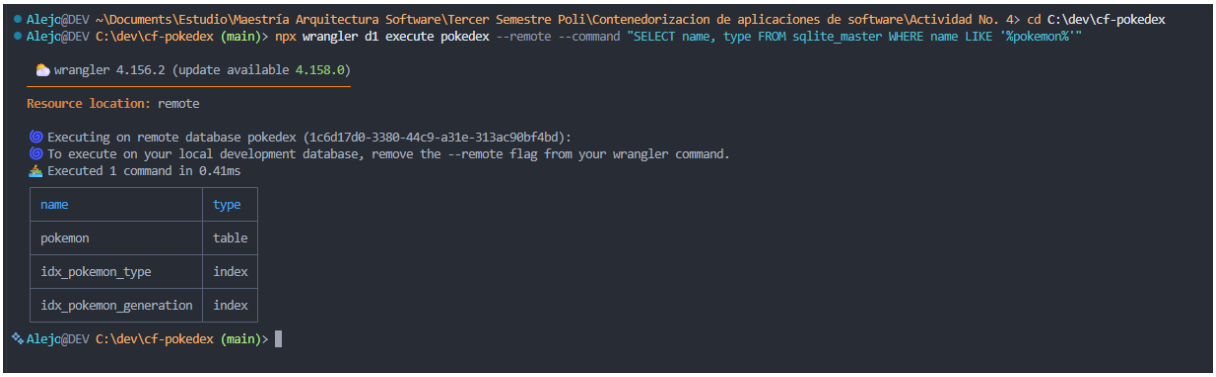


Figura 3. Estructura creada en la base de datos remota: la tabla pokemon y sus dos índices.



Figura 4. Consultas SQL ejecutadas sobre la base y filas leídas por cada una.



Figura 5. Verificación de la carga en la base de la nube: 1215 filas y 1025 IDs distintos.

5. APLICACIÓN WEB SERVERLESS

La aplicación cumple lo pedido: muestra los datos en **tabla, tarjeta y gráficas**, y permite buscar por **ID** (clave principal) y por **nombre** (columna secundaria), con filtros por tipo y generación. El navegador solo habla con la API del Worker; nunca con la base de datos.

5.1 Backend: un Worker con Chain of Responsibility

Tabla 5. Endpoints de la API.

Endpoint	Qué hace
`GET /api/health`	Estado del servicio
`GET /api/pokemon/{id}`	Todas las formas de una especie (búsqueda por ID), con la forma Base primero
`GET /api/pokemon?name=pika`	Búsqueda parcial por nombre, sin distinguir mayúsculas

Endpoint	Qué hace
`GET /api/pokemon?type=Fire&generation=1&limit=25&cursor=…`	Listado filtrado, ordenado por ID y paginado con cursor
`GET /api/stats`	Totales, conteo por tipo y generación, promedios y los cinco mayores

Decisiones de diseño que sostienen el patrón serverless:

- **Cadena creada una vez por aislado.** El repositorio y la cadena de manejadores se crean al primer uso y se reutilizan en las peticiones siguientes.
- **Paginación por cursor.** Se pide una fila de más para saber con certeza si hay otra página; el cursor codifica la última clave leída. Con ello el orden es global por `(id, form)` y no se usa `OFFSET`.
- **Validación y errores claros.** Tipo, generación, límite (máximo 100), ID y cursor se validan; responden 400, 404 o 405 con un mensaje en JSON, nunca con la traza interna.
- **Consultas parametrizadas.** Todo valor del usuario va como parámetro enlazado, y los comodines de `LIKE` se escapan para que `%` o _ no se interpreten.
- **Solo lectura y sin credenciales en el código.** El Worker solo hace `SELECT` y accede a la base por su enlace `DB`; no hay claves que filtrar.
- **Logs estructurados.** Una línea JSON por petición, visible con `wrangler tail`.

5.2 Frontend: JavaScript puro, sin dependencias

HTML, CSS y JavaScript sin librerías externas (las gráficas son HTML y SVG), por lo que no hay cadena de suministro de terceros que mantener. Solo se piden tipografías a Google Fonts, con alternativa del sistema si no cargan.

- **Explorador:** buscador con modo «Por ID» y «Por nombre», filtros, tabla paginada con «Cargar más», y una **ficha** del registro con su número grande, tipos, barras de estadísticas, gráfica radial y selector de formas.
- **Panorama:** total de registros, especies únicas, promedio, barras por tipo, columnas por generación, promedios por estadística y los cinco con mayor total (cada uno abre su ficha).
- **Accesibilidad:** etiquetas en formularios, foco visible, tabla navegable con teclado, descripciones en las gráficas y respeto de `prefers-reduced-motion` y del modo oscuro.

Las Figuras 6, 7 y 8 muestran la aplicación publicada en https://pokedex-serverless.pokedex-serverless.workers.dev: la búsqueda por ID (el 6 devuelve a Charizard con sus tres formas, sus barras de estadísticas y el gráfico radial), la búsqueda por nombre (`pika` devuelve las dos formas de Pikachu) y el filtro combinado por tipo y generación (Poison, generación 7: 8 registros, con la ficha de Alolan Grimer).

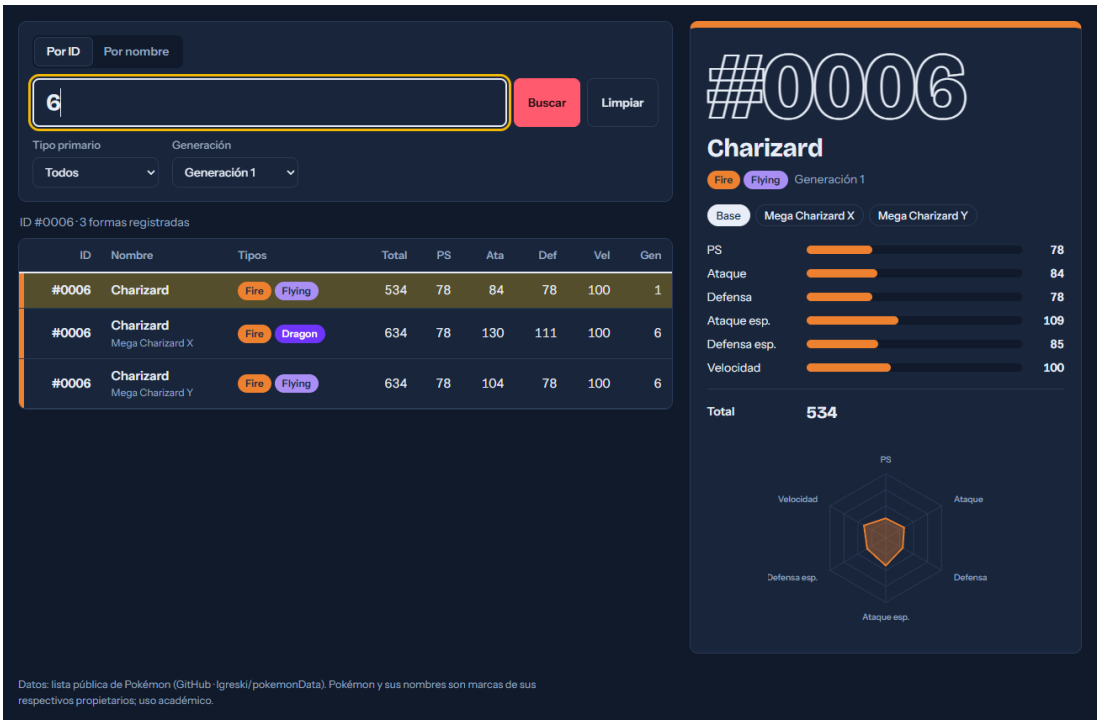


Figura 6. Aplicación publicada: búsqueda por ID (6) con la ficha de Charizard y sus tres formas.

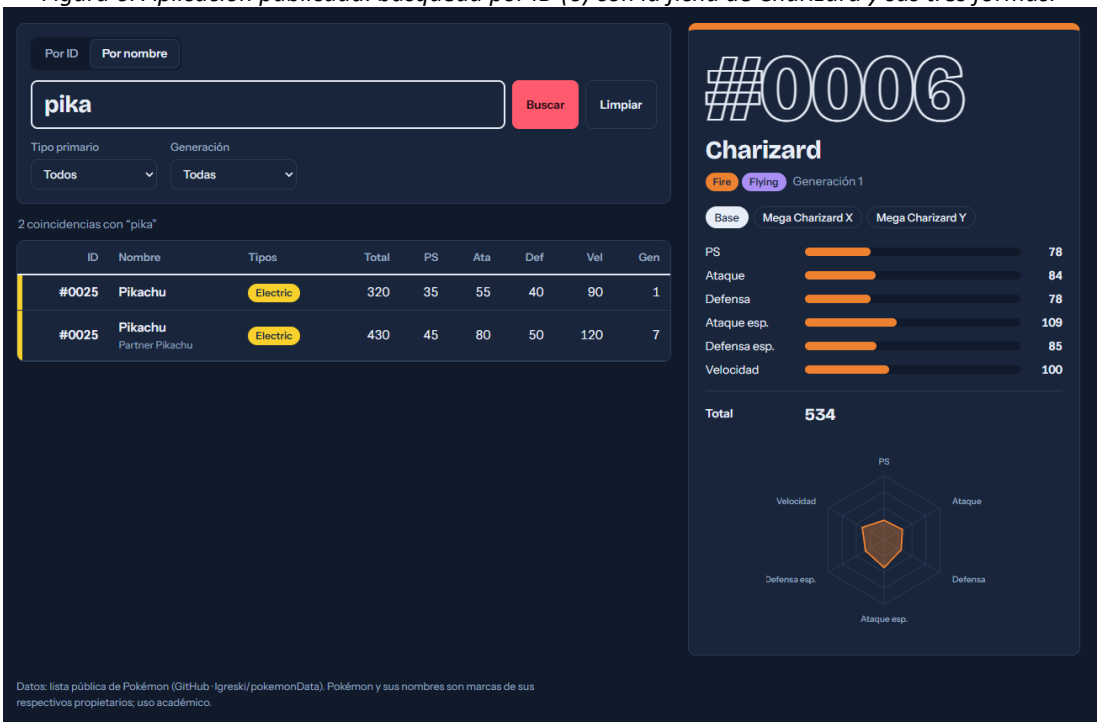


Figura 7. Aplicación publicada: búsqueda por nombre (pika). La ficha conserva el registro seleccionado antes.

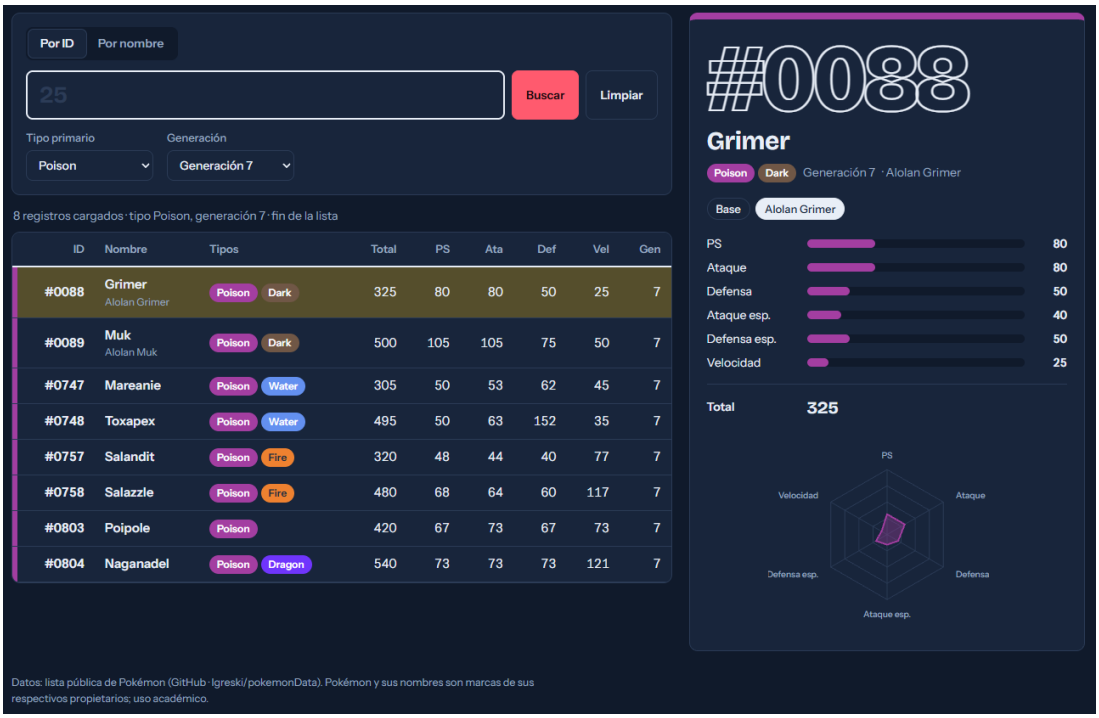


Figura 8. Aplicación publicada: filtro por tipo (Poison) y generación (7), con la ficha de Alolan Grimer.

6. DESPLIEGUE EN CLOUDFLARE

El despliegue de la aplicación se hace por completo desde la terminal con wrangler, la herramienta de línea de comandos de Cloudflare, y no requiere aprovisionar ningún servidor. El proceso crea la base de datos D1, carga en ella el esquema y los 1.215 registros, verifica que la carga esté completa y publica el Worker junto con los archivos del sitio en una dirección pública. Todo se ejecutó con una cuenta del plan gratuito, sin método de pago. En esta sección se describen los pasos, la comprobación de la API publicada, las dificultades encontradas y la evidencia del despliegue.

6.1 Requisitos y pasos

Requisitos: Node.js 18 o superior, una cuenta gratuita de Cloudflare (plan Workers Free, sin método de pago) y `wrangler`, que se instala con `npm install`. La URL pública resultante es https://pokedex-serverless.pokedex-serverless.workers.dev.

- **Conectar la cuenta.** `npx wrangler login` abre el navegador para autorizar a Wrangler; `npx wrangler whoami` confirma la sesión y que el token tiene permiso de escritura sobre Workers y D1.
- **Probar en local.** `npm run db:local` crea el esquema y carga los 1.215 registros en una D1 local, y `npm run dev` levanta la aplicación en `http://localhost:8787`.
- **Crear la base en la nube.** `npx wrangler d1 create pokedex` crea la base y entrega su `database_id`, que se escribe en `wrangler.toml`.
- **Cargar esquema y datos.** `npx wrangler d1 execute pokedex --remote --file db/schema.sql` y luego con `db/seed.sql`.
- **Verificar la carga.** Un `SELECT COUNT(*)` con `--remote` debe dar 1.215 filas y 1.025 IDs distintos.
- **Publicar.** `npx wrangler deploy` sube el Worker y los archivos del frontend, y deja la base enlazada como `env.DB`.

```
npm install
npx wrangler login
npm run db:local
npm run dev
npx wrangler d1 create pokedex
npx wrangler d1 execute pokedex --remote --file db/schema.sql
npx wrangler d1 execute pokedex --remote --file db/seed.sql
npx wrangler d1 execute pokedex --remote --command "SELECT COUNT(*) AS filas, COUNT(DISTINCT id) AS
ids FROM pokemon"
npx wrangler deploy
```

6.2 Comprobación de la API publicada

Una vez publicada la aplicación, se comprobó que la API respondiera desde internet y no solo en el entorno local. Para ello se consultaron cuatro rutas con curl sobre la dirección pública del Worker: el estado del servicio, la búsqueda por ID, la búsqueda por nombre y las estadísticas. Las cuatro respondieron con estado 200 y devolvieron los datos esperados, entre ellos la ficha de Pikachu con sus dos formas, lo que confirma que el Worker llega a la base de datos D1 y que la carga de los 1.215 registros quedó completa:

```
curl https://pokedex-serverless.pokedex-serverless.workers.dev/api/health
curl https://pokedex-serverless.pokedex-serverless.workers.dev/api/pokemon/25
curl "https://pokedex-serverless.pokedex-serverless.workers.dev/api/pokemon?name=pika"
curl https://pokedex-serverless.pokedex-serverless.workers.dev/api/stats
```

Para no dejar recursos activos al terminar, el repositorio incluye `scripts/destroy.sh`, que elimina el Worker y la base de datos. Este script no fue probado de extremo a extremo; si falla, el Worker y la base D1 también se pueden eliminar desde el panel de Cloudflare.

6.3 Dificultades encontradas

Durante el despliegue y las pruebas aparecieron tres dificultades, ninguna de las cuales exigió cambiar el diseño de la aplicación ni comprometió el resultado final. Dos tuvieron que ver con el entorno (una ruta con tildes en Windows y la espera inicial del certificado del subdominio) y una con un aviso informativo de wrangler. Se documentan porque pueden repetirse en otro equipo y su solución es sencilla.

- **Rutas con tildes o espacios en Windows.** En una carpeta como `...\Maestría Arquitectura Software\...`, los comandos con `--local` fallaban con un error interno, incluso un `SELECT 1`. En una ruta sin tildes (`C:\dev\cf-pokedex`) funcionaron sin cambiar nada, lo que indica que el problema es el motor local de `wrangler` con esa ruta y no el código. Solución: trabajar desde una carpeta con ruta simple.
- **Primer acceso a la URL.** Justo después del despliegue, la URL pública no conectó durante unos 2 minutos, mientras Cloudflare preparaba el certificado del subdominio nuevo. Después respondió con normalidad.
- **Aviso de subdominio `workers.dev`.** Durante el despliegue `wrangler` avisó que faltaba registrar un subdominio, pero publicó igualmente con `pokedex-serverless`; se puede cambiar en el panel de Cloudflare (Workers → Subdomain).

6.4 Evidencia del despliegue

La evidencia del despliegue se reúne en tres capturas que muestran, desde tres ángulos distintos, que la aplicación quedó publicada y funcionando en Cloudflare sin ningún servidor propio. La Figura 9 corresponde a la salida de wrangler deploy: allí se ve que el comando leyó los cuatro

archivos del sitio desde la carpeta frontend, que el Worker quedó enlazado a la base de datos mediante el enlace env.DB hacia la base D1 pokedex, y que el despliegue terminó con la dirección pública de la aplicación y el identificador de la versión publicada. Este último dato permite identificar con exactitud qué código está en producción.

Las Figuras 10 y 11 muestran el Worker ya en funcionamiento. La Figura 10 presenta su página en el panel de Cloudflare, donde aparecen el enlace DB hacia la base pokedex, los registros del Worker activados, las invocaciones recibidas sin ningún error y el historial de despliegues realizados con Wrangler. La Figura 11 presenta los registros en tiempo real obtenidos con wrangler tail: cada petición a la API queda registrada como una línea JSON con su método, su ruta, sus parámetros, su estado 200 y su duración, que en esta captura va desde 0 hasta 66 milisegundos. En conjunto, estas evidencias demuestran el despliegue, el acceso a la base de datos y la observabilidad del servicio, que son los puntos centrales de un despliegue serverless.

Estas capturas complementan la comprobación de la sección 6.2: allí se verificó con curl que las rutas de la API respondían con estado 200, y aquí se muestra, desde el propio Cloudflare, el mismo resultado visto de otra manera, como el enlace a la base de datos, las invocaciones sin errores y el registro de cada petición. El panel del Worker, además, muestra tres despliegues hechos con Wrangler a lo largo de la práctica, todos en estado «Ready», lo que evidencia que el despliegue es repetible: publicar una nueva versión es volver a ejecutar un solo comando. Por último, en las capturas del panel de Cloudflare se taparon los datos personales de la cuenta, como el correo electrónico.

Toda esta evidencia puede reproducirse por cualquier persona que clone el repositorio y tenga una cuenta gratuita de Cloudflare: bastan los pasos de la sección 6.1, que terminan en un solo comando de publicación. La observabilidad no requirió instalar ninguna herramienta adicional, porque los registros del Worker se activan con la opción observability del archivo wrangler.toml y se consultan en tiempo real con wrangler tail, una conexión temporal que se abre únicamente mientras se necesita. Todo el proceso se realizó con el plan gratuito y sin método de pago registrado en la cuenta, lo que muestra que un despliegue serverless completo, con base de datos, función, sitio y registros, es alcanzable sin costo ni administración de infraestructura para una aplicación de este tamaño.

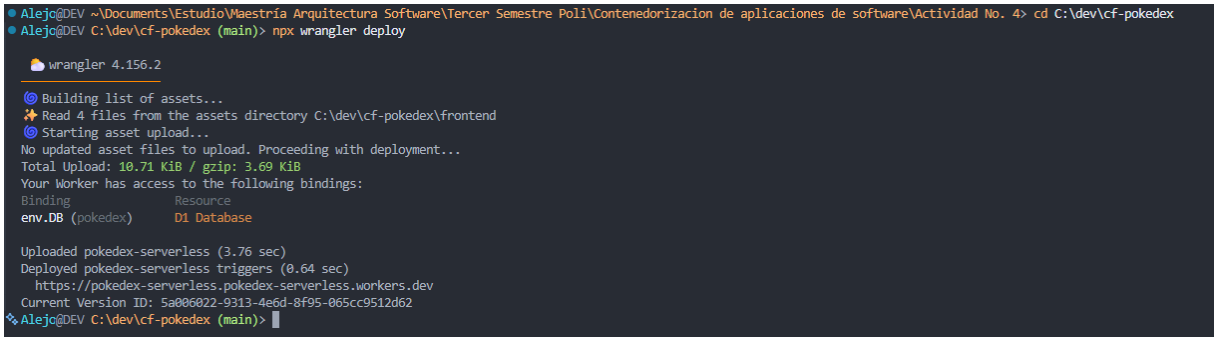


Figura 9. Salida del despliegue con wrangler deploy: Worker publicado y enlazado a la base D1.

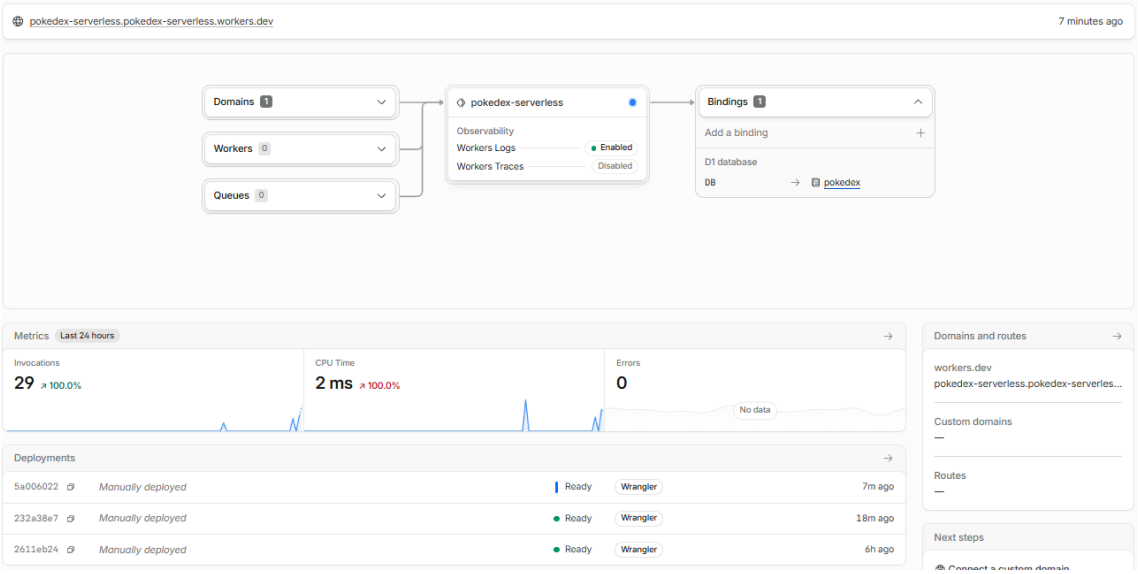


Figura 10. Worker pokedex-serverless en el panel de Cloudflare: enlace DB a la base D1 pokedex, registros activos, invocaciones y despliegues.

```
Alejo@DEV C:\dev\cf-pokedex (main)> npx wrangler tail pokedex-serverless

 wrangler 4.156.2

Successfully created tail, expires at 2026-11-03T08:14:23Z
Connected to pokedex-serverless, waiting for logs...
GET https://pokedex-serverless.pokedex-serverless.workers.dev/api/pokemon?limit=25&generation=1 - Ok @ 2/11/2026, 9:15:44 p. m.
  (log) {"method":"GET","path":"/pokemon","query":{"limit":"25","generation":"1"},"status":200,"duration_ms":62}
GET https://pokedex-serverless.pokedex-serverless.workers.dev/api/health - Ok @ 2/11/2026, 9:15:44 p. m.
  (log) {"method":"GET","path":"/health","query":{},"status":200,"duration_ms":0}
GET https://pokedex-serverless.pokedex-serverless.workers.dev/api/pokemon/1 - Ok @ 2/11/2026, 9:15:44 p. m.
  (log) {"method":"GET","path":"/pokemon/1","query":{},"status":200,"duration_ms":14}
GET https://pokedex-serverless.pokedex-serverless.workers.dev/api/pokemon/25 - Ok @ 2/11/2026, 9:15:50 p. m.
  (log) {"method":"GET","path":"/pokemon/25","query":{},"status":200,"duration_ms":12}
GET https://pokedex-serverless.pokedex-serverless.workers.dev/api/pokemon?limit=25 - Ok @ 2/11/2026, 9:16:46 p. m.
  (log) {"method":"GET","path":"/pokemon","query":{"limit":"25"},"status":200,"duration_ms":15}
GET https://pokedex-serverless.pokedex-serverless.workers.dev/api/pokemon?limit=25&type=Poison - Ok @ 2/11/2026, 9:16:52 p. m.
  (log) {"method":"GET","path":"/pokemon","query":{"limit":"25","type":"Poison"},"status":200,"duration_ms":18}
GET https://pokedex-serverless.pokedex-serverless.workers.dev/api/pokemon?limit=25&type=Poison&generation=7 - Ok @ 2/11/2026, 9:16:55 p. m.
  (log) {"method":"GET","path":"/pokemon","query":{"limit":"25","type":"Poison","generation":"7"},"status":200,"duration_ms":16}
GET https://pokedex-serverless.pokedex-serverless.workers.dev/api/health - Ok @ 2/11/2026, 9:18:40 p. m.
  (log) {"method":"GET","path":"/health","query":{},"status":200,"duration_ms":0}
GET https://pokedex-serverless.pokedex-serverless.workers.dev/api/pokemon?limit=25&generation=1 - Ok @ 2/11/2026, 9:18:40 p. m.
  (log) {"method":"GET","path":"/pokemon","query":{"limit":"25","generation":"1"},"status":200,"duration_ms":66}
GET https://pokedex-serverless.pokedex-serverless.workers.dev/api/pokemon/1 - Ok @ 2/11/2026, 9:18:40 p. m.
  (log) {"method":"GET","path":"/pokemon/1","query":{},"status":200,"duration_ms":16}
GET https://pokedex-serverless.pokedex-serverless.workers.dev/api/pokemon?limit=25&type=Poison&generation=1 - Ok @ 2/11/2026, 9:18:50 p. m.
  (log) {"method":"GET","path":"/pokemon","query":{"limit":"25","type":"Poison","generation":"1"},"status":200,"duration_ms":14}
GET https://pokedex-serverless.pokedex-serverless.workers.dev/api/pokemon?limit=25&type=Poison&generation=7 - Ok @ 2/11/2026, 9:18:52 p. m.
  (log) {"method":"GET","path":"/pokemon","query":{"limit":"25","type":"Poison","generation":"7"},"status":200,"duration_ms":13}
GET https://pokedex-serverless.pokedex-serverless.workers.dev/api/pokemon/88 - Ok @ 2/11/2026, 9:19:02 p. m.
  (log) {"method":"GET","path":"/pokemon/88","query":{},"status":200,"duration_ms":11}
```

Figura 11. Registros estructurados del Worker en tiempo real (wrangler tail): ruta, consulta, estado y duración de cada petición.

7. PRUEBAS Y RESULTADOS

La aplicación quedó **publicada y respondiendo en https://pokedex-serverless.pokedex-serverless.workers.dev**, con 1.215 registros verificados en la base de la nube. Antes de publicar se ejecutaron 23 pruebas automáticas y 19 comprobaciones de la interfaz.

Tabla 6. Resumen de pruebas y resultados.

Qué se verificó	Pruebas	Resultado
Worker contra una base SQLite real: búsqueda por ID, filtros, paginación, búsqueda por nombre, escape de comodines, validaciones 400/404/405, estadísticas, caché	17	Aprobadas
Generador de SQL: tipos, vacíos a `NULL`, colisión de Zygarde, clave duplicada, SQL ejecutable e idempotente	6	Aprobadas

Qué se verificó	Pruebas	Resultado
Interfaz sobre el Worker local (motor `workerd` y D1 local): búsqueda por ID y nombre, filtros, «Cargar más», ficha, panorama y saltos entre vistas, en un navegador simulado	19	Aprobadas
Aplicación publicada: `/api/health`, `/api/pokemon/25`, `/api/pokemon?name=pika`, `/api/stats` y la página principal con sus archivos	5	Todas con estado 200

Resultados concretos:

- La base tiene **1.215 filas y 1.025 IDs distintos**, igual en la D1 local y en la de la nube, y la carga es idempotente.
- `/api/pokemon/25` devuelve a Pikachu con sus dos formas: Base (generación 1) y Partner Pikachu (generación 7).
- `/api/pokemon?name=pika` encuentra 2 coincidencias sin distinguir mayúsculas.
- `/api/stats` informa 1.215 registros, 1.025 especies y el conteo por tipo.
- Recorrer los 150 registros de tipo Water en páginas de 7 devuelve cada uno una sola vez y en orden de ID.
- Un ID inexistente responde 404 y un `%` escrito en el buscador no se interpreta como comodín.

```
npm test
# 17 pruebas del Worker + 6 del generador de SQL: 23 aprobadas
```

Limitación declarada: las comprobaciones de la interfaz se hicieron en un navegador simulado; la verificación visual en un navegador real se documenta con las capturas de las secciones 4, 5 y 6.

8. VIDEO EXPLICATIVO

El video, de **33 minutos y 13 segundos**, muestra la pantalla con narración y recorre las tres actividades en el mismo orden de la rúbrica: carga a la nube, despliegue serverless y análisis de lo realizado. La Tabla 7 resume sus partes y el criterio que cubre cada una.

- **Enlace público del video:** https://youtu.be/TodbzdBe8oU
- **Nota de visualización.** El video debe reproducirse en calidad **1080p HD** (en YouTube: icono de engranaje, opción «Calidad» y luego «1080p»). En calidades inferiores el texto del código y de las terminales no se lee correctamente.

Tabla 7. Partes del video y criterio de la rúbrica que cubre cada una.

Parte	Qué se muestra	Criterio de la rúbrica
1	Objetivo, flujo completo y patrón elegido	Análisis de lo realizado
2	Dataset: columnas, tipos y hallazgos de calidad de datos	Carga de información a la base de datos
3	D1 en el panel de Cloudflare: tabla, claves e índices; carga y verificación 1.215 = 1.215 con `--remote`	Carga de información a la base de datos
4	Arquitectura: Worker, D1 y archivos estáticos, qué hace cada uno y por qué; `wrangler.toml` y el despliegue	Despliegue de aplicaciones serverless
5	Demo en la URL pública: búsqueda por ID y por nombre, filtros, ficha, panorama y registros en vivo	Despliegue de aplicaciones serverless
6	Por qué Cloudflare y por qué el patrón serverless, con ventajas y límites; cierre	Análisis de lo realizado

9. CONCLUSIONES

Una aplicación de lectura con tráfico intermitente se despliega completa en Cloudflare sin administrar un solo servidor y sin tarjeta de pago; ese es el valor práctico del patrón serverless frente a mantener un clúster de contenedores. En la Unidad 3 el despliegue exigía crear el clúster, dimensionar los nodos y operar su escalado; aquí el mismo resultado, una aplicación pública con su base de datos, se obtuvo con un archivo de configuración y un comando de publicación, y no consume recursos mientras nadie la usa. Esa diferencia no significa que serverless sea siempre mejor, pero sí que para una aplicación de este tamaño elimina por completo el trabajo de infraestructura.

- El modelo de datos manda. El ID repetido del dataset obligó a una clave compuesta (id y form), y el análisis previo de la calidad de los datos evitó perder registros: dos filas de Zygarde chocaban en la misma clave y se resolvieron sin descartarlas, de modo que las 1.215 filas del archivo llegaron completas a la base. Elegir bien las claves y los índices decidió qué consultas leen pocas filas, como la búsqueda por ID o los filtros por tipo y generación, y cuáles recorren la tabla, como la búsqueda parcial por nombre.
- Los patrones se componen. El despliegue serverless resuelve dónde y cómo corre el código; Chain of Responsibility ordena el código dentro del Worker. Cada ruta de la API tiene su propio manejador con una sola responsabilidad, de modo que agregar un endpoint es agregar una clase, sin modificar las existentes. Los dos patrones no compiten: uno define la forma de desplegar y el otro la forma de organizar lo desplegado.
- Probar en el entorno real importa. Las 23 pruebas automáticas pasaban, pero al abrir la aplicación publicada en un navegador apareció un error en la ficha que la prueba simulada no había detectado: en lugar de las barras de estadísticas se mostraba texto sin sentido. Se corrigió, se agregó una prueba que lo cubre y se volvió a desplegar. La lección es que las pruebas automáticas reducen el riesgo, pero no reemplazan la revisión del resultado final donde lo ve el usuario.
- El entorno también cuenta. Una ruta con tildes en Windows rompió el motor local de wrangler sin que el código tuviera errores; documentarlo evita que le pase a otra persona que clone el repositorio en una carpeta similar. Del mismo modo, la espera inicial del certificado del subdominio es un comportamiento normal de la plataforma, no una falla.
- Límites reconocidos. La búsqueda parcial por nombre recorre la tabla; con unos 1.200 registros es aceptable, pero con millones se reemplazaría por un índice de búsqueda dedicado. La aplicación usa un único Worker para toda la API; si creciera, cada grupo de rutas podría desplegarse por separado. Además, el plan gratuito tiene topes diarios de lectura y escritura, que esta aplicación no se acerca a alcanzar con su tráfico actual, y cuando se superan las consultas se rechazan en lugar de cobrarse.
- Evolución natural. El siguiente paso sería agregar autenticación para poder escribir datos, un dominio propio en lugar de la dirección workers.dev, un pipeline de despliegue continuo que publique cada cambio desde el repositorio y alertas sobre los registros del Worker. Ninguno de estos cambios exige rediseñar la aplicación, porque la separación entre el sitio estático, la función y la base de datos ya está resuelta.

En conjunto, la actividad muestra que el patrón serverless reduce la infraestructura a lo esencial (una base de datos, una función y un sitio) y traslada el esfuerzo del equipo a lo que realmente importa: modelar bien los datos, ordenar el código y verificar que el resultado funcione donde lo usa la persona final.

10. BIBLIOGRAFÍA

Las fuentes que siguen se presentan en formato APA y en orden alfabético. Incluyen la lectura fundamental de la Unidad 4, de la que se tomaron los patrones aplicados en la aplicación (el despliegue serverless y Chain of Responsibility); los trabajos académicos sobre computación serverless y microservicios que esa lectura referencia; la documentación de Cloudflare consultada para los servicios y los planes utilizados; y el repositorio público de donde se obtuvo el conjunto de datos.

Baldini, I., Castro, P., Chang, K., Cheng, P., Fink, S., Ishakian, V., … & Suter, P. (2017). Serverless computing: Current trends and open problems. En S. Chaudhary, G. Somani & R. Buyya (Eds.), *Research advances in cloud computing* (pp. 1-20). Springer. https://doi.org/10.1007/978-981-10-5026-8_1

Cloudflare. (s. f.). *Pricing: Workers*. Cloudflare Developers. https://developers.cloudflare.com/workers/about/pricing

Dragoni, N., Lanese, I., Larsen, S. T., Mazzara, M., Mustafin, R., & Safina, L. (2017). Microservices: How to make your application scale. En W. Abramowicz & R. Corchuelo (Eds.), *Advances in service-oriented and cloud computing* (pp. 95-104). Springer. https://doi.org/10.1007/978-3-319-67262-5_8

Lgreski. (s. f.). *pokemonData* [Conjunto de datos, `Pokemon.csv`]. GitHub. https://github.com/lgreski/pokemonData

Pedraza Bonilla, V. A. (s. f.). *Lectura fundamental. Unidad 4: Patrones*. Módulo Contenerización de Aplicaciones de Software. Politécnico Grancolombiano.

Villamizar, M., Garcés, O., Castro, H., Verano, M., Salamanca, L., Casallas, R., & Gil, S. (2016). Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud. *2015 10th Computing Colombian Conference (10CCC)*. https://doi.org/10.1109/ColumbianCC.2015.7333476

11. ANEXO A: ENTREGA DEL CÓDIGO Y EJECUCIÓN

El código completo y las instrucciones de ejecución se encuentran en el repositorio https://github.com/AlejoTechEngineer/pokedex-serverless. Su estructura es la siguiente:

Tabla 8. Estructura del repositorio.

Archivo	Contenido
`db/schema.sql`	Script de base de datos: tabla `pokemon` con clave `(id, form)` e índices por tipo y generación
`db/seed.sql`	Datos: 1.215 `INSERT OR REPLACE`, generados desde el CSV
`data/pokemon.csv`	Dataset original
`scripts/build_seed.py`	Convierte el CSV en `seed.sql`: infiere tipos, limpia vacíos, resuelve colisiones y valida claves
`scripts/deploy.sh` y `scripts/destroy.sh`	Atajos de despliegue y eliminación (no probados de extremo a extremo)
`src/worker.js`	Punto de entrada del Worker
`src/chain.js`	Manejadores del patrón Chain of Responsibility
`src/repository.js`	Consultas SQL a D1, paginación por cursor y estadísticas
`frontend/`	Aplicación web: `index.html`, `styles.css`, `app.js`, `config.js`
`wrangler.toml`	Configuración de Cloudflare: Worker, archivos estáticos y enlace a D1
`tests/`	23 pruebas automáticas: 17 del Worker y 6 del generador de SQL

Secuencia de ejecución:

```
npm install
npm run db:local        # esquema y 1.215 registros en una D1 local
npm run dev             # http://localhost:8787
npm test                # 17 pruebas del Worker + 6 del generador de SQL
npx wrangler login      # publicar en la nube: ver sección 6
```

12. ANEXO B: CÓDIGO FUENTE

Se transcriben los archivos principales del repositorio. En `wrangler.toml` el valor `database_id` se reemplaza por el identificador real que entrega `wrangler d1 create pokedex`.

B.1 wrangler.toml

El archivo wrangler.toml concentra toda la configuración del despliegue: es el único lugar donde se declara qué se publica y con qué recursos se conecta. Define el nombre del Worker (pokedex-serverless), su archivo de entrada (src/worker.js) y la fecha de compatibilidad del entorno de ejecución. La sección [assets] indica que la carpeta frontend se sirve como archivos estáticos y que solo las rutas /api/* ejecutan el Worker. La sección [[d1_databases]] enlaza la base de datos pokedex con el nombre DB, que es el que usa el código para consultarla. Por último, [observability] activa los registros del Worker. En el repositorio, el valor de database_id es el identificador real que entrega wrangler d1 create pokedex; aquí se muestra un texto de reemplazo.

```
# Configuración de Cloudflare Workers.
name = "pokedex-serverless"
main = "src/worker.js"
compatibility_date = "2025-09-01"

# Frontend estático: lo sirve Cloudflare. Solo /api/* ejecuta el Worker.
[assets]
directory = "./frontend"
run_worker_first = ["/api/*"]

# Base de datos serverless D1. database_id lo escribe scripts/deploy.sh tras crearla.
# En local (wrangler dev) el valor no importa.
[[d1_databases]]
binding = "DB"
database_name = "pokedex"
database_id = "REEMPLAZAR_CON_ID_REAL"

[observability]
enabled = true
```

B.2 db/schema.sql

El archivo db/schema.sql es el script de la base de datos: crea la tabla pokemon y sus dos índices secundarios. La clave primaria es compuesta, formada por id y form, porque el ID del dataset se repite en las especies con formas alternativas, y la tabla se declara WITHOUT ROWID para que se almacene ordenada por esa clave. Además de las columnas del archivo original, incluye name_lower, una copia del nombre en minúsculas que permite buscar sin distinguir mayúsculas, y deja type2 como columna opcional, porque no todos los registros tienen un segundo tipo. Los índices idx_pokemon_type e idx_pokemon_generation resuelven los filtros por tipo y por generación. Todas las sentencias usan IF NOT EXISTS, así que el script puede ejecutarse varias veces sin error, y se aplica con wrangler d1 execute pokedex --remote --file db/schema.sql.

```
-- Esquema de la base de datos serverless (Cloudflare D1, SQLite).
-- Clave primaria compuesta (id, form): el ID del dataset se repite en 153 especies por
-- sus formas alternas (Mega, Alola, Galar...), pero la pareja (id, form) es única.
-- WITHOUT ROWID: la tabla se almacena ordenada por su clave primaria.

CREATE TABLE IF NOT EXISTS pokemon (
  id          INTEGER NOT NULL,
  form        TEXT    NOT NULL,            -- "Base" cuando el CSV viene vacío
  name        TEXT    NOT NULL,
  name_lower  TEXT    NOT NULL,            -- para búsqueda sin distinguir mayúsculas
  type1       TEXT    NOT NULL,
  type2       TEXT,                        -- NULL en los de un solo tipo
  total       INTEGER NOT NULL,
  hp          INTEGER NOT NULL,
  attack      INTEGER NOT NULL,
  defense     INTEGER NOT NULL,
  sp_atk      INTEGER NOT NULL,
  sp_def      INTEGER NOT NULL,
  speed       INTEGER NOT NULL,
  generation  INTEGER NOT NULL,
  PRIMARY KEY (id, form)
) WITHOUT ROWID;

-- Índices secundarios: filtrar por tipo o generación ya ordenado por id.
CREATE INDEX IF NOT EXISTS idx_pokemon_type       ON pokemon (type1, id);
CREATE INDEX IF NOT EXISTS idx_pokemon_generation ON pokemon (generation, id);
```

B.3 scripts/build_seed.py

El script scripts/build_seed.py convierte el archivo CSV en el script SQL de carga db/seed.sql. Lee el CSV, normaliza los nombres de las columnas a minúsculas con guion bajo e infiere el tipo de cada columna observando todos sus valores, de modo que se conservan la estructura y los tipos del dataset. Las celdas vacías se convierten en NULL, salvo la forma, que toma el valor «Base» porque forma parte de la clave primaria. También detecta las colisiones de clave, como las dos filas de Zygarde con el mismo ID y la forma vacía, y las resuelve usando el nombre en lugar de descartar registros. Antes de escribir nada valida que cada pareja (id, form) sea única y genera 1.215 sentencias INSERT OR REPLACE en lotes de 50, por lo que cargar el archivo varias veces no duplica datos. Se ejecuta con python scripts/build_seed.py.

```
#!/usr/bin/env python3
"""Convierte el CSV en un script SQL de carga para Cloudflare D1 (db/seed.sql).

Qué hace
1. Lee el CSV (UTF-8, con o sin BOM) y normaliza los nombres de columna a snake_case.
2. Infiere el tipo de CADA columna mirando todos sus valores (entero, decimal o texto).
3. Limpia espacios; celdas vacías -> NULL; Form vacío -> "Base".
4. Resuelve colisiones de clave (id, form) usando el nombre (p. ej. Zygarde 50% / 10%).
5. Valida que (id, form) sea único y genera INSERT OR REPLACE por lotes (idempotente).

Uso
    python scripts/build_seed.py                       # escribe db/seed.sql
    python scripts/build_seed.py --csv otro.csv --out /tmp/seed.sql
"""
from __future__ import annotations

import argparse
import csv
import re
import sys
from collections import Counter
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
COLUMNS = ["id", "form", "name", "name_lower", "type1", "type2", "total", "hp", "attack",
           "defense", "sp_atk", "sp_def", "speed", "generation"]
```

```
BATCH = 50
_INT = re.compile(r"^-?\d+$")
_FLOAT = re.compile(r"^-?\d+\.\d+$")



def snake_case(header: str) -> str:
    """'Sp. Atk' -> 'sp_atk'."""
    return re.sub(r"[^a-z0-9]+", "_", header.strip().lower()).strip("_")



def infer_types(rows: list[dict], cols: list[str]) -> dict[str, str]:
    types = {}
    for c in cols:
        vals = [r[c] for r in rows if r[c] != ""]
        if vals and all(_INT.match(v) for v in vals):
            types[c] = "int"
        elif vals and all(_INT.match(v) or _FLOAT.match(v) for v in vals):
            types[c] = "float"
        else:
            types[c] = "str"
    return types



def build_rows(csv_path: str | Path) -> tuple[list[dict], dict]:
    with open(csv_path, newline="", encoding="utf-8-sig") as fh:
        reader = csv.DictReader(fh)
        cols = [snake_case(h) for h in (reader.fieldnames or [])]
        rows = [{snake_case(k): (v or "").strip() for k, v in r.items() if k is not None}
                for r in reader]
    missing = {"id", "name", "type1", "total", "generation"} - set(cols)
    if missing:
        raise ValueError(f"Faltan columnas en el CSV: {sorted(missing)}")

    types = infer_types(rows, cols)
    items = []
    for r in rows:
        it = {}
        for c in cols:
            v = r[c]
            if v == "":
                it[c] = None
            elif types[c] == "int":
                it[c] = int(v)
            elif types[c] == "float":
                it[c] = float(v)
            else:
                it[c] = v
        it["form"] = it.get("form") or "Base"
        it["name_lower"] = it["name"].lower()
        items.append(it)

    # Colisiones (id, form): se desambiguan con el nombre.
    counts = Counter((i["id"], i["form"]) for i in items)
    resolved = []
    for i in items:
        if counts[(i["id"], i["form"])] > 1:
            i["form"] = i["name"]
            resolved.append(f"{i['id']}:{i['name']}")
    dupes = [k for k, n in Counter((i["id"], i["form"]) for i in items).items() if n > 1]
    if dupes:
        raise ValueError(f"Clave (id, form) duplicada sin resolver: {dupes[:3]}")

    report = {
        "rows": len(items),
        "columns": types,
        "distinct_ids": len({i["id"] for i in items}),
        "collisions_resolved": resolved,
        "blank_form": sum(1 for r in rows if r.get("form", "") == ""),
        "blank_type2": sum(1 for r in rows if r.get("type2", "") == ""),
    }
```

```
    return items, report


def sql_value(v) -> str:
    if v is None:
        return "NULL"
    if isinstance(v, (int, float)):
        return repr(v)
    return "'" + str(v).replace("'", "''") + "'"


def render_sql(items: list[dict]) -> str:
    head = ("-- Generado por scripts/build_seed.py. No editar a mano.\n"
            "-- INSERT OR REPLACE: se puede ejecutar varias veces sin duplicar datos.\n")
    cols = ", ".join(COLUMNS)
    out = [head]
    for i in range(0, len(items), BATCH):
        chunk = items[i:i + BATCH]
        values = ",\n  ".join("(" + ", ".join(sql_value(it.get(c)) for c in COLUMNS) + ")"
                              for it in chunk)
        out.append(f"INSERT OR REPLACE INTO pokemon ({cols}) VALUES\n  {values};\n")
    return "\n".join(out)


def main(argv=None) -> int:
    p = argparse.ArgumentParser(description="CSV -> SQL de carga para D1")
    p.add_argument("--csv", default=str(ROOT / "data" / "pokemon.csv"))
    p.add_argument("--out", default=str(ROOT / "db" / "seed.sql"))
    args = p.parse_args(argv)

    items, rep = build_rows(args.csv)
    Path(args.out).write_text(render_sql(items), encoding="utf-8")
    print(f"· CSV leído: {rep['rows']} filas · {rep['distinct_ids']} valores distintos de id")
    print("· Tipos inferidos:", rep["columns"])
    if rep["collisions_resolved"]:
        print("· Colisiones (id, form) resueltas con el nombre:", rep["collisions_resolved"])
    print(f"· Celdas vacías: form={rep['blank_form']} (-> 'Base'), type2={rep['blank_type2']} (->
NULL)")
    print(f"· Escrito {args.out} con {rep['rows']} filas en {-(-rep['rows'] // BATCH)} lotes")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

B.4 src/worker.js

El archivo src/worker.js es el punto de entrada del Worker: recibe cada petición a /api/*, la entrega a la cadena de manejadores y devuelve la respuesta. La cadena de manejadores se crea una sola vez por instancia del Worker y se reutiliza en las peticiones siguientes, asociada a la conexión con la base de datos, en lugar de reconstruirse cada vez. La función handleRequest traduce la petición, la procesa y captura los errores: los errores esperados, como una ruta inexistente o un dato inválido, se convierten en una respuesta con su estado y un mensaje en JSON, y cualquier otro error devuelve un estado 500 sin exponer detalles internos. Al terminar, escribe en el registro una línea JSON con el método, la ruta, los parámetros, el estado y la duración, que se consulta en tiempo real con wrangler tail.

```
// Punto de entrada del Worker.
//
// La cadena de manejadores se crea una sola vez por "isolate" y se reutiliza en las
// peticiones siguientes (arranque en caliente). Los archivos estáticos del
// frontend los sirve Cloudflare directamente (ver [assets] en wrangler.toml); este Worker
// solo recibe las rutas /api/*.
```

```
import { ApiError, buildChain, jsonResponse, parseRequest } from "./chain.js";
import { PokemonRepository } from "./repository.js";

const chains = new WeakMap(); // env.DB -> cadena

function chainFor(env) {
  if (!chains.has(env.DB)) chains.set(env.DB, buildChain(new PokemonRepository(env.DB)));
  return chains.get(env.DB);
}

export async function handleRequest(request, env) {
  const started = Date.now();
  const req = parseRequest(request);
  let response;
  try {
    response = await chainFor(env).handle(req);
  } catch (err) {
    if (err instanceof ApiError) {
      response = jsonResponse(err.status, { error: err.message });
    } else {
      console.error("Error no controlado", err);
      response = jsonResponse(500, { error: "Error interno del servidor." });
    }
  }
  // Log estructurado (JSON): se ve en tiempo real con `wrangler tail`.
  console.log(JSON.stringify({
    method: req.method, path: req.path, query: Object.fromEntries(req.query),
    status: response.status, duration_ms: Date.now() - started,
  }));
  return response;
}

export default {
  fetch: (request, env) => handleRequest(request, env),
};
```

B.5 src/chain.js

El archivo src/chain.js implementa el patrón Chain of Responsibility, con el que el Worker resuelve cada ruta de la API. La clase base Handler define el mecanismo: cada eslabón decide con matches si la petición es suya y, si lo es, la atiende con process; si no, la pasa al siguiente, y si ninguno la atiende responde un error 404. Sobre esa base hay cinco manejadores con una sola responsabilidad cada uno: MethodGuardHandler acepta solo el método GET y responde la verificación previa del navegador, HealthHandler informa el estado del servicio, PokemonByIdHandler busca por ID, PokemonListHandler lista con filtros y paginación, y StatsHandler entrega las estadísticas. La función buildChain los enlaza en ese orden. El archivo también valida cada dato recibido (el ID, el tipo, la generación y el límite, que no puede pasar de 100) y construye las respuestas JSON con sus cabeceras de caché; agregar un endpoint nuevo consiste en escribir una clase y enlazarla, sin modificar las existentes.

```
// Enrutamiento con el patrón Chain of Responsibility (Unidad 4, sección 2).
//
// Cada manejador tiene UNA responsabilidad: reconocer su ruta y atenderla, o delegar al
// siguiente eslabón. Agregar un endpoint = agregar una clase y enlazarla (abierto/cerrado).
//
//   MethodGuard → Health → PokemonById → PokemonList → Stats → (fin: 404)

import { InvalidCursor } from "./repository.js";

export const VALID_TYPES = new Set([
  "Bug", "Dark", "Dragon", "Electric", "Fairy", "Fighting", "Fire", "Flying", "Ghost",
  "Grass", "Ground", "Ice", "Normal", "Poison", "Psychic", "Rock", "Steel", "Water",
```

```
]);
const MAX_LIMIT = 100;
const DEFAULT_LIMIT = 20;

export class ApiError extends Error {
  constructor(status, message) {
    super(message);
    this.status = status;
  }
}

export function jsonResponse(status, body = null, cacheSeconds = 0) {
  return new Response(body === null ? null : JSON.stringify(body), {
    status,
    headers: {
      "Content-Type": "application/json; charset=utf-8",
      "Cache-Control": cacheSeconds ? `public, max-age=${cacheSeconds}` : "no-store",
      "X-Content-Type-Options": "nosniff",
    },
  });
}

/** Convierte un Request de Workers en {method, path, query}; quita el prefijo /api. */
export function parseRequest(request) {
  const url = new URL(request.url);
  const path = url.pathname.replace(/^\/api(?=\/|$)/, "").replace(/\/+$/, "") || "/";
  return { method: request.method.toUpperCase(), path, query: url.searchParams };
}

function intParam(raw, name, min, max = null) {
  if (raw === null || raw === "") return null;
  if (!/^\d{1,9}$/.test(raw)) throw new ApiError(400, `'${name}' debe ser un entero positivo.`);
  const value = Number(raw);
  if (value < min || (max !== null && value > max)) {
    throw new ApiError(400, `'${name}' debe estar ${max !== null ? `entre ${min} y ${max}` : `≥
${min}`}.`);
  }
  return value;
}

// ---------------------------------------------------------------- eslabón base
class Handler {
  #next = null;
  setNext(handler) {
    this.#next = handler;
    return handler; // permite encadenar: a.setNext(b).setNext(c)
  }
  async handle(req) {
    if (this.matches(req)) return this.process(req);
    if (this.#next) return this.#next.handle(req);
    throw new ApiError(404, `Ruta no encontrada: ${req.method} ${req.path}`);
  }
  matches() { throw new Error("not implemented"); }
  process() { throw new Error("not implemented"); }
}

// ---------------------------------------------------------------- manejadores
export class MethodGuardHandler extends Handler {
  matches(req) { return req.method !== "GET"; }
  process(req) {
    if (req.method === "OPTIONS") return jsonResponse(204);
    throw new ApiError(405, "Método no permitido. Esta API es de solo lectura (GET).");
  }
}

export class HealthHandler extends Handler {
  matches(req) { return req.path === "/health"; }
  process() { return jsonResponse(200, { status: "ok" }); }
}
```

```
export class PokemonByIdHandler extends Handler {
  static ROUTE = /^\/pokemon\/([^/]+)$/;
  constructor(repo) { super(); this.repo = repo; }
  matches(req) { return PokemonByIdHandler.ROUTE.test(req.path); }
  async process(req) {
    const id = intParam(PokemonByIdHandler.ROUTE.exec(req.path)[1], "id", 1);
    const forms = await this.repo.getForms(id);
    if (!forms.length) throw new ApiError(404, `No existe un registro con id ${id}.`);
    return jsonResponse(200, { id, forms }, 300);
  }
}

export class PokemonListHandler extends Handler {
  constructor(repo) { super(); this.repo = repo; }
  matches(req) { return req.path === "/pokemon"; }
  async process(req) {
    const q = req.query;
    let type = null;
    if (q.has("type") && q.get("type") !== "") {
      const t = q.get("type").trim();
      type = t.charAt(0).toUpperCase() + t.slice(1).toLowerCase();
      if (!VALID_TYPES.has(type)) {
        throw new ApiError(400, `Tipo inválido. Opciones: ${[...VALID_TYPES].sort().join(", ")}.`);
      }
    }
    const generation = intParam(q.get("generation"), "generation", 1, 99);
    const limit = intParam(q.get("limit"), "limit", 1, MAX_LIMIT) ?? DEFAULT_LIMIT;
    const name = (q.get("name") ?? "").trim().slice(0, 40) || null;
    try {
      const page = await this.repo.listPage({ type, generation, name, limit, cursor: q.get("cursor")
|| null });
      return jsonResponse(200, { count: page.items.length, ...page }, 60);
    } catch (err) {
      if (err instanceof InvalidCursor) throw new ApiError(400, err.message);
      throw err;
    }
  }
}

export class StatsHandler extends Handler {
  constructor(repo) { super(); this.repo = repo; }
  matches(req) { return req.path === "/stats"; }
  async process() { return jsonResponse(200, await this.repo.stats(), 60); }
}

export function buildChain(repo) {
  const head = new MethodGuardHandler();
  head
    .setNext(new HealthHandler())
    .setNext(new PokemonByIdHandler(repo))
    .setNext(new PokemonListHandler(repo))
    .setNext(new StatsHandler(repo));
  return head;
}
```

B.6 src/repository.js

El archivo src/repository.js es la capa de acceso a datos: concentra todas las consultas SQL a la base D1, de modo que el resto del código no conoce la estructura de las tablas. La clase PokemonRepository ofrece tres operaciones. getForms busca todas las formas de una especie por su ID y deja la forma «Base» primero. listPage devuelve una página de registros filtrada por tipo, por generación o por nombre; los filtros por tipo y generación aprovechan los índices secundarios, y la búsqueda por nombre usa LIKE sobre la columna en minúsculas, con los comodines escapados para que los símbolos % y _ escritos por la persona no se interpreten. La paginación usa un cursor basado en la última clave leída y pide una fila de más para saber con certeza si existe otra página, sin usar OFFSET. Por último, stats calcula los totales, la cantidad por tipo y por generación, los promedios y los cinco registros con mayor total mediante cuatro consultas agregadas ejecutadas en un solo lote, y

guarda el resultado 60 segundos para no repetirlo en cada petición. Todos los valores del usuario viajan como parámetros enlazados, nunca concatenados en el texto de la consulta.

```
// Capa de acceso a datos (Cloudflare D1). Sin lógica HTTP: solo consultas SQL.
//
// Estrategia de consulta
//  * por id         -> clave primaria (id, form)
//  * por tipo       -> índice idx_pokemon_type       (type1, id)
//  * por generación -> índice idx_pokemon_generation (generation, id)
//  * por nombre     -> LIKE sobre name_lower (tabla pequeña: ~1.2k filas)
// Paginación por cursor ("keyset"): estable y sin OFFSET, con orden global por (id, form).

export const STAT_FIELDS = ["hp", "attack", "defense", "sp_atk", "sp_def", "speed"];
const COLUMNS =
  "id, form, name, type1, type2, total, hp, attack, defense, sp_atk, sp_def, speed, generation";
const STATS_TTL_MS = 60_000;

export class InvalidCursor extends Error {}

function toBase64Url(text) {
  const bytes = new TextEncoder().encode(text);
  let bin = "";
  for (const b of bytes) bin += String.fromCharCode(b);
  return btoa(bin).replace(/\+/g, "-").replace(/\//g, "_").replace(/=+$/, "");
}

function fromBase64Url(token) {
  const padded = token.replace(/-/g, "+").replace(/_/g, "/") + "=".repeat((4 - (token.length % 4)) %
4);
  const bin = atob(padded);
  return new TextDecoder().decode(Uint8Array.from(bin, (c) => c.charCodeAt(0)));
}

export function encodeCursor(row) {
  return toBase64Url(JSON.stringify({ id: row.id, form: row.form }));
}

export function decodeCursor(token) {
  try {
    const c = JSON.parse(fromBase64Url(token));
    if (!Number.isInteger(c.id) || typeof c.form !== "string" || c.form.length > 120) throw new
Error();
    return c;
  } catch {
    throw new InvalidCursor("El cursor de paginación no es válido.");
  }
}

// null (p. ej. type2 de un solo tipo) se omite para mantener el JSON compacto.
const publicRow = (row) => Object.fromEntries(Object.entries(row).filter(([, v]) => v !== null));
const round1 = (n) => Math.round(n * 10) / 10;

export class PokemonRepository {
  constructor(db) {
    this.db = db;
    this.statsCache = null;
  }

  async getForms(id) {
    const { results } = await this.db
      .prepare(
        `SELECT ${COLUMNS} FROM pokemon WHERE id = ?
         ORDER BY CASE WHEN form = 'Base' THEN 0 ELSE 1 END, form`,
      )
      .bind(id)
      .all();
    return results.map(publicRow);
  }
```

```
  async listPage({ type, generation, name, limit, cursor }) {
    const where = [];
    const params = [];
    if (name) {
      // Se escapan los comodines de LIKE que escriba la persona.
      where.push("name_lower LIKE ? ESCAPE '\\'");
      params.push(`%${name.toLowerCase().replace(/[\\%_]/g, (m) => "\\" + m)}%`);
    }
    if (type) { where.push("type1 = ?"); params.push(type); }
    if (generation != null) { where.push("generation = ?"); params.push(generation); }
    const filter = where.length ? "WHERE " + where.join(" AND ") : "";

    if (name) {
      // Búsqueda por nombre: primeras `limit` coincidencias + total de coincidencias.
      const [page, count] = await this.db.batch([
        this.db.prepare(`SELECT ${COLUMNS} FROM pokemon ${filter} ORDER BY id, form LIMIT
?`).bind(...params, limit),
        this.db.prepare(`SELECT COUNT(*) AS n FROM pokemon ${filter}`).bind(...params),
      ]);
      return { items: page.results.map(publicRow), next_cursor: null, total_matches:
count.results[0].n };
    }

    const keyset = [...where];
    const keyParams = [...params];
    if (cursor) {
      const c = decodeCursor(cursor);
      keyset.push("(id > ? OR (id = ? AND form > ?))");
      keyParams.push(c.id, c.id, c.form);
    }
    const sql = `SELECT ${COLUMNS} FROM pokemon ${keyset.length ? "WHERE " + keyset.join(" AND ") :
""}
                 ORDER BY id, form LIMIT ?`;
    // Se pide una fila de más para saber con certeza si existe otra página.
    const { results } = await this.db.prepare(sql).bind(...keyParams, limit + 1).all();
    const hasMore = results.length > limit;
    const rows = hasMore ? results.slice(0, limit) : results;
    return {
      items: rows.map(publicRow),
      next_cursor: hasMore ? encodeCursor(rows[rows.length - 1]) : null,
    };
  }

  async stats() {
    const now = Date.now();
    if (this.statsCache && now - this.statsCache.at < STATS_TTL_MS) return this.statsCache.value;

    const avg = [...STAT_FIELDS, "total"].map((f) => `AVG(${f}) AS avg_${f}`).join(", ");
    const [totals, byType, byGen, top] = await this.db.batch([
      this.db.prepare(`SELECT COUNT(*) AS records, COUNT(DISTINCT id) AS species, ${avg} FROM
pokemon`),
      this.db.prepare("SELECT type1 AS type, COUNT(*) AS count FROM pokemon GROUP BY type1 ORDER BY
count DESC, type1"),
      this.db.prepare("SELECT generation, COUNT(*) AS count FROM pokemon GROUP BY generation ORDER
BY generation"),
      this.db.prepare("SELECT id, name, form, total FROM pokemon ORDER BY total DESC, id, form LIMIT
5"),
    ]);
    const t = totals.results[0];
    const value = {
      total_records: t.records,
      unique_species: t.species,
      by_type: byType.results,
      by_generation: byGen.results,
      average_stats: Object.fromEntries(STAT_FIELDS.map((f) => [f, round1(t[`avg_${f}`])])),
      average_total: round1(t.avg_total),
      top_total: top.results,
    };
    this.statsCache = { at: now, value };
    return value;
  }
```

```
}
```

B.7 frontend/index.html

El archivo frontend/index.html es la estructura de la página: contiene solo el contenido y deja los estilos y el comportamiento en archivos aparte. Tiene un encabezado con el título, el estado de la API y las dos pestañas de navegación («Explorador» y «Panorama»). La vista «Explorador» reúne el buscador, con sus modos «Por ID» y «Por nombre», los filtros por tipo y por generación, la tabla de resultados con el botón «Cargar más» y el espacio donde se dibuja la ficha del registro seleccionado; la vista «Panorama» queda vacía y se llena con las estadísticas que entrega la API. La página incluye elementos de accesibilidad: un enlace para saltar al contenido, etiquetas asociadas a cada campo, regiones que anuncian los cambios a los lectores de pantalla y un título descriptivo para la tabla. Carga las tipografías desde Google Fonts, con una alternativa del sistema si no están disponibles, y al final enlaza config.js, que indica la dirección de la API, y app.js, que dibuja los datos y atiende las acciones de la persona.

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Pokédex Serverless · Explorador de datos</title>
  <meta name="description" content="Aplicación web serverless que consulta un dataset de 1.215
registros almacenado en Cloudflare D1.">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link rel="stylesheet"
href="https://fonts.googleapis.com/css2?family=Bricolage+Grotesque:opsz,wght@12..96,500;12..96,700;1
2..96,800&family=Instrument+Sans:wght@400;500;600&display=swap">
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <a class="skip" href="#main">Saltar al contenido</a>

  <header class="top">
    <div class="top__title">
      <h1>Pokédex Serverless</h1>
      <p>1.215 registros leídos en vivo desde Cloudflare D1, a través de un Worker serverless.</p>
    </div>
    <div class="top__right">
      <div class="status" id="apiStatus" role="status" aria-live="polite">
        <span class="status__dot" aria-hidden="true"></span><span id="apiStatusText">Conectando con
la API…</span>
      </div>
      <div class="tabs" role="tablist" aria-label="Vistas">
        <button class="tab is-active" role="tab" id="tab-explore" aria-selected="true" aria-
controls="view-explore">Explorador</button>
        <button class="tab" role="tab" id="tab-overview" aria-selected="false" aria-controls="view-
overview">Panorama</button>
      </div>
    </div>
  </header>

  <main id="main">
    <div class="banner" id="banner" role="alert" hidden></div>

    <!-- ============================ EXPLORADOR ============================ -->
    <section id="view-explore" role="tabpanel" aria-labelledby="tab-explore" class="explore">
      <div class="explore__list">
        <form class="console" id="searchForm" autocomplete="off">
          <div class="mode" role="radiogroup" aria-label="Buscar por">
            <label class="mode__opt"><input type="radio" name="mode" value="id" checked><span>Por
ID</span></label>
            <label class="mode__opt"><input type="radio" name="mode" value="name"><span>Por
nombre</span></label>
```

```
          </div>
          <div class="console__row">
            <label class="sr-only" for="q" id="qLabel">Número de la Pokédex</label>
            <input id="q" class="console__input" type="text" inputmode="numeric" placeholder="25"
maxlength="40" required>
            <button class="btn btn--primary" type="submit">Buscar</button>
            <button class="btn" type="button" id="clearBtn">Limpiar</button>
          </div>
          <div class="filters">
            <label>Tipo primario
              <select id="fType"><option value="">Todos</option></select>
            </label>
            <label>Generación
              <select id="fGen">
                <option value="">Todas</option>
              </select>
            </label>
          </div>
        </form>

        <p class="summary" id="summary" aria-live="polite"></p>

        <div class="table-wrap">
          <table class="grid" id="grid">
            <caption class="sr-only">Registros del dataset</caption>
            <thead>
              <tr>
                <th scope="col" class="num">ID</th>
                <th scope="col">Nombre</th>
                <th scope="col">Tipos</th>
                <th scope="col" class="num">Total</th>
                <th scope="col" class="num">PS</th>
                <th scope="col" class="num">Ata</th>
                <th scope="col" class="num">Def</th>
                <th scope="col" class="num">Vel</th>
                <th scope="col" class="num">Gen</th>
              </tr>
            </thead>
            <tbody id="rows"></tbody>
          </table>
          <p class="empty" id="empty" hidden></p>
        </div>
        <button class="btn btn--wide" id="moreBtn" hidden>Cargar más registros</button>
      </div>

      <aside class="entry" id="entry" aria-live="polite" aria-label="Ficha del registro
seleccionado"></aside>
    </section>

    <!-- ============================ PANORAMA ============================ -->
    <section id="view-overview" role="tabpanel" aria-labelledby="tab-overview" hidden
class="overview"></section>
  </main>

  <footer class="foot">
    <p>Datos: lista pública de Pokémon (GitHub · lgreski/pokemonData). Pokémon y sus nombres son
marcas de sus respectivos propietarios; uso académico.</p>
  </footer>

  <script src="config.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

B.8 frontend/styles.css

El archivo frontend/styles.css define toda la apariencia de la aplicación, sin librerías externas. Empieza con las variables de diseño (colores, tipografías y radio de los bordes), que se redefinen para

el modo oscuro según la preferencia del sistema de la persona. La disposición del explorador usa una cuadrícula de dos columnas, con la tabla a la izquierda y la ficha a la derecha, que se apila en una sola columna en pantallas estrechas; la tabla mantiene fijo su encabezado y se desplaza dentro de su propio contenedor, para que la página nunca se desborde. La ficha destaca el número del registro con un contorno grande, dibuja las barras de estadísticas y el gráfico radial con el color del tipo y los chips de tipo toman su color de la misma paleta. La vista «Panorama» organiza sus tarjetas en una cuadrícula de doce columnas. El archivo cuida además la accesibilidad: un contorno amarillo visible al navegar con el teclado, una clase para texto solo para lectores de pantalla y la desactivación de las transiciones cuando la persona pide menos movimiento.

```
:root {
  --ground: #dde4ed;
  --panel: #ffffff;
  --ink: #1b2a41;
  --muted: #566579;
  --line: #c6d0dd;
  --accent: #d7263d;
  --accent-ink: #ffffff;
  --signal: #f2b705;
  --ok: #1f8a4c;
  --bad: #b3261e;
  --radius: 10px;
  --font-display: "Bricolage Grotesque", "Segoe UI", system-ui, sans-serif;
  --font-body: "Instrument Sans", "Segoe UI", system-ui, sans-serif;
}

@media (prefers-color-scheme: dark) {
  :root {
    --ground: #101a2b;
    --panel: #18253b;
    --ink: #e8eef7;
    --muted: #9aaac0;
    --line: #2c3c57;
    --accent: #ff5a6e;
    --accent-ink: #1b0a0d;
    --ok: #4cc884;
    --bad: #ff8a80;
  }
}

* { box-sizing: border-box; }
html { color-scheme: light dark; }
body {
  margin: 0;
  background: var(--ground);
  color: var(--ink);
  font: 400 15px/1.5 var(--font-body);
}
h1, h2, h3, .big-num { font-family: var(--font-display); }
.sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0);
white-space: nowrap; }
.skip { position: absolute; left: -999px; top: 8px; background: var(--ink); color: var(--panel);
padding: 8px 12px; border-radius: 6px; z-index: 10; }
.skip:focus { left: 8px; }
:focus-visible { outline: 3px solid var(--signal); outline-offset: 2px; }

/* ---------------------------------------------------------------- cabecera */
.top {
  display: flex; flex-wrap: wrap; gap: 16px 32px; align-items: flex-end; justify-content: space-
between;
  padding: 24px clamp(16px, 4vw, 48px) 16px;
  border-bottom: 4px solid var(--accent);
  background: var(--panel);
}
.top h1 { margin: 0; font-size: clamp(28px, 4vw, 40px); line-height: 1.05; font-weight: 800; letter-
spacing: -0.02em; }
.top p { margin: 6px 0 0; color: var(--muted); max-width: 52ch; }
.top__right { display: flex; flex-direction: column; align-items: flex-end; gap: 12px; }
```

```
.status { display: flex; align-items: center; gap: 8px; font-size: 13px; color: var(--muted); }
.status__dot { width: 9px; height: 9px; border-radius: 50%; background: var(--signal); }
.status.is-ok .status__dot { background: var(--ok); }
.status.is-bad .status__dot { background: var(--bad); }
.tabs { display: flex; gap: 4px; background: var(--ground); padding: 4px; border-radius: 999px; }
.tab {
  border: 0; background: transparent; color: var(--muted); font: 600 14px var(--font-body);
  padding: 8px 18px; border-radius: 999px; cursor: pointer;
}
.tab.is-active { background: var(--ink); color: var(--panel); }

main { padding: 20px clamp(16px, 4vw, 48px) 8px; max-width: 1360px; margin: 0 auto; }
.banner { background: var(--bad); color: #fff; padding: 12px 16px; border-radius: var(--radius);
margin-bottom: 16px; }
.banner[hidden], [hidden] { display: none !important; }

/* ---------------------------------------------------------------- explorador */
.explore { display: grid; gap: 20px; grid-template-columns: 1fr; align-items: start; }
@media (min-width: 1000px) { .explore { grid-template-columns: minmax(0, 1.25fr) minmax(340px,
0.75fr); } }

.console { background: var(--panel); border-radius: var(--radius); padding: 16px; border: 1px solid
var(--line); display: grid; gap: 12px; }
.mode { display: inline-flex; background: var(--ground); border-radius: 8px; padding: 3px; justify-
self: start; }
.mode__opt { position: relative; }
.mode__opt input { position: absolute; opacity: 0; inset: 0; cursor: pointer; }
.mode__opt span { display: block; padding: 6px 14px; border-radius: 6px; font-weight: 600; font-
size: 14px; color: var(--muted); }
.mode__opt input:checked + span { background: var(--panel); color: var(--ink); box-shadow: 0 0 0 1px
var(--line); }
.mode__opt input:focus-visible + span { outline: 3px solid var(--signal); outline-offset: 2px; }

.console__row { display: flex; gap: 8px; flex-wrap: wrap; }
.console__input {
  flex: 1 1 200px; min-width: 0; font: 700 28px/1 var(--font-display); letter-spacing: -0.01em;
  padding: 10px 14px; border-radius: 8px; border: 2px solid var(--ink); background: var(--panel);
color: var(--ink);
}
.console__input::placeholder { color: var(--line); }
.btn {
  font: 600 14px var(--font-body); padding: 10px 16px; border-radius: 8px; cursor: pointer;
  border: 1px solid var(--line); background: var(--panel); color: var(--ink);
}
.btn:hover { border-color: var(--ink); }
.btn--primary { background: var(--accent); border-color: var(--accent); color: var(--accent-ink); }
.btn--primary:hover { filter: brightness(0.93); border-color: var(--accent); }
.btn--wide { width: 100%; margin-top: 12px; padding: 12px; }
.btn[disabled] { opacity: 0.6; cursor: progress; }

.filters { display: flex; gap: 16px; flex-wrap: wrap; }
.filters label { display: grid; gap: 4px; font-size: 13px; color: var(--muted); font-weight: 500; }
.filters select { font: 500 14px var(--font-body); padding: 7px 10px; border-radius: 8px; border:
1px solid var(--line); background: var(--panel); color: var(--ink); min-width: 150px; }

.summary { margin: 14px 2px 8px; color: var(--muted); font-size: 14px; min-height: 1.5em; }

.table-wrap { background: var(--panel); border: 1px solid var(--line); border-radius: var(--radius);
overflow: auto; max-height: 62vh; }
table.grid { width: 100%; border-collapse: collapse; font-variant-numeric: tabular-nums; }
.grid th { position: sticky; top: 0; background: var(--panel); text-align: left; font-size: 13px;
color: var(--muted); font-weight: 600; padding: 10px 12px; border-bottom: 2px solid var(--ink);
white-space: nowrap; z-index: 1; }
.grid td { padding: 9px 12px; border-bottom: 1px solid var(--line); white-space: nowrap; }
.grid .num { text-align: right; }
.grid tbody tr { cursor: pointer; }
.grid tbody td:first-child { box-shadow: inset 6px 0 0 var(--type, transparent); }
.grid tbody tr:hover td { background: color-mix(in srgb, var(--ink) 6%, transparent); }
.grid tbody tr.is-selected td { background: color-mix(in srgb, var(--signal) 28%, transparent); }
.grid tbody tr:focus-visible { outline-offset: -3px; }
.grid td:first-child { padding-left: 20px; font-weight: 600; }
```

```
.cell-name { font-weight: 600; }
.cell-form { display: block; font-size: 12px; color: var(--muted); font-weight: 400; }
.empty { padding: 28px 16px; text-align: center; color: var(--muted); margin: 0; }

.chips { display: inline-flex; gap: 4px; }
.chip { display: inline-block; font-size: 12px; font-weight: 600; padding: 2px 9px; border-radius:
999px; }

/* ---------------------------------------------------------------------- ficha */
.entry { background: var(--panel); border: 1px solid var(--line); border-radius: var(--radius);
padding: 20px; position: sticky; top: 16px; border-top: 8px solid var(--type, var(--ink)); }
.entry__placeholder { color: var(--muted); margin: 0; }
.big-num { font-size: clamp(64px, 11vw, 112px); line-height: 0.9; font-weight: 800; margin: 0;
letter-spacing: -0.04em; color: transparent; -webkit-text-stroke: 2px var(--ink); font-variant-
numeric: tabular-nums; }
.entry h2 { margin: 6px 0 0; font-size: 30px; line-height: 1.1; letter-spacing: -0.01em; }
.entry__meta { display: flex; flex-wrap: wrap; gap: 8px; align-items: center; margin: 10px 0 18px;
color: var(--muted); font-size: 14px; }
.forms { display: flex; flex-wrap: wrap; gap: 6px; margin: 0 0 18px; }
.forms button { font: 500 13px var(--font-body); border: 1px solid var(--line); background: var(--
panel); color: var(--ink); padding: 4px 10px; border-radius: 999px; cursor: pointer; }
.forms button[aria-pressed="true"] { background: var(--ink); color: var(--panel); border-color:
var(--ink); }

.stat { display: grid; grid-template-columns: 108px 1fr 40px; align-items: center; gap: 10px;
margin: 7px 0; font-size: 14px; }
.stat__bar { height: 10px; background: var(--ground); border-radius: 999px; overflow: hidden; }
.stat__fill { display: block; height: 100%; background: var(--type, var(--ink)); border-radius:
999px; }
.stat__val { text-align: right; font-weight: 700; font-variant-numeric: tabular-nums; }
.stat--total { margin-top: 14px; padding-top: 12px; border-top: 1px solid var(--line); font-weight:
700; }
.stat--total .stat__bar { display: none; }
.stat--total { grid-template-columns: 108px 1fr; }
.stat--total .stat__val { text-align: left; font-size: 20px; font-family: var(--font-display); }

.radar { display: block; width: 100%; max-width: 280px; margin: 18px auto 0; }
.radar text { fill: var(--muted); font: 500 11px var(--font-body); }
.radar .ring { fill: none; stroke: var(--line); stroke-width: 1; }
.radar .axis { stroke: var(--line); stroke-width: 1; }
.radar .shape { fill: color-mix(in srgb, var(--type, var(--ink)) 35%, transparent); stroke: var(--
type, var(--ink)); stroke-width: 2; }

/* ---------------------------------------------------------------------- panorama */
.overview { display: grid; gap: 20px; grid-template-columns: repeat(12, 1fr); }
.card { background: var(--panel); border: 1px solid var(--line); border-radius: var(--radius);
padding: 18px 20px; grid-column: span 12; }
.card h2 { margin: 0 0 14px; font-size: 20px; letter-spacing: -0.01em; }
@media (min-width: 900px) { .card--half { grid-column: span 6; } .card--third { grid-column: span 4;
} }
.kpi { margin: 0; }
.kpi dd { margin: 0; font: 800 clamp(36px, 5vw, 56px)/1 var(--font-display); letter-spacing: -
0.03em; font-variant-numeric: tabular-nums; }
.kpi dt { color: var(--muted); font-size: 14px; margin-top: 6px; }
.hbar { display: grid; grid-template-columns: 84px 1fr 44px; gap: 10px; align-items: center; margin:
5px 0; font-size: 14px; }
.hbar__track { height: 14px; background: var(--ground); border-radius: 4px; overflow: hidden; }
.hbar__fill { display: block; height: 100%; background: var(--c, var(--ink)); }
.hbar__val { text-align: right; font-variant-numeric: tabular-nums; font-weight: 600; }
.cols { display: flex; align-items: flex-end; gap: 8px; height: 200px; padding-top: 8px; }
.cols__item { flex: 1; display: flex; flex-direction: column; justify-content: flex-end; align-
items: center; height: 100%; gap: 4px; font-size: 13px; }
.cols__bar { width: 100%; background: var(--ink); border-radius: 4px 4px 0 0; min-height: 2px; }
.cols__n { font-weight: 700; font-variant-numeric: tabular-nums; }
.top5 { list-style: none; margin: 0; padding: 0; }
.top5 li { display: flex; justify-content: space-between; gap: 12px; padding: 9px 0; border-bottom:
1px solid var(--line); }
.top5 li:last-child { border-bottom: 0; }
.top5 b { font-variant-numeric: tabular-nums; }
.top5 button { all: unset; cursor: pointer; font-weight: 600; text-decoration: underline; text-
decoration-color: var(--line); text-underline-offset: 3px; }
.top5 button:focus-visible { outline: 3px solid var(--signal); outline-offset: 2px; }
```

```
.foot { padding: 20px clamp(16px, 4vw, 48px) 32px; color: var(--muted); font-size: 12.5px; max-
width: 1360px; margin: 0 auto; }
.foot p { margin: 0; max-width: 80ch; }

@media (prefers-reduced-motion: no-preference) {
  .stat__fill, .hbar__fill, .cols__bar { transition: width 0.35s ease, height 0.35s ease; }
}
```

B.9 frontend/app.js

El archivo frontend/app.js es el cliente de la aplicación: pide los datos a la API del Worker y los dibuja en la página, sin librerías externas. Una función auxiliar construye los elementos de la página de forma segura, sin insertar texto como código, y otra concentra las llamadas a la API con mensajes de error claros. La tabla se llena con los registros de cada consulta y admite la paginación con el botón «Cargar más». La búsqueda por ID consulta todas las formas de la especie, y la búsqueda por nombre amplía los filtros a todas las generaciones, salvo que la persona haya elegido filtros por su cuenta. La ficha del registro muestra el número, los tipos, las seis estadísticas como barras, el total y un gráfico radial dibujado en SVG, con un selector de formas cuando la especie tiene varias. La pestaña «Panorama» carga las estadísticas de la API y dibuja las barras por tipo, las columnas por generación y los cinco registros con mayor total, cada uno enlazado a su ficha. Al iniciar, comprueba el estado de la API y muestra el tiempo de respuesta.

```
/* Pokédex Serverless · cliente web (JavaScript puro, sin dependencias).
 * Habla con la API (Cloudflare Worker + D1) y pinta tabla, ficha y gráficas.   */
(() => {
  "use strict";

  const API = String((window.APP_CONFIG || {}).apiUrl || "").replace(/\/$/, "");

  const TYPE_COLORS = {
    Normal: "#a8a77a", Fire: "#ee8130", Water: "#6390f0", Electric: "#f7d02c", Grass: "#7ac74c",
    Ice: "#96d9d6", Fighting: "#c22e28", Poison: "#a33ea1", Ground: "#e2bf65", Flying: "#a98ff3",
    Psychic: "#f95587", Bug: "#a6b91a", Rock: "#b6a136", Ghost: "#735797", Dragon: "#6f35fc",
    Dark: "#705746", Steel: "#b7b7ce", Fairy: "#d685ad",
  };
  const STATS = [
    ["hp", "PS"], ["attack", "Ataque"], ["defense", "Defensa"],
    ["sp_atk", "Ataque esp."], ["sp_def", "Defensa esp."], ["speed", "Velocidad"],
  ];
  const STAT_MAX = 255;
  const PAGE_SIZE = 25;

  const $ = (id) => document.getElementById(id);
  const ui = {
    form: $("searchForm"), q: $("q"), qLabel: $("qLabel"), type: $("fType"), gen: $("fGen"),
    rows: $("rows"), empty: $("empty"), more: $("moreBtn"), summary: $("summary"),
    entry: $("entry"), banner: $("banner"), clear: $("clearBtn"),
    apiStatus: $("apiStatus"), apiStatusText: $("apiStatusText"), overview: $("view-overview"),
  };

  const state = {
    mode: "id", items: [], next: null, selected: null, view: "explore",
    overviewLoaded: false, formsCache: new Map(), busy: false, filtersTouched: false,
  };

  /* ---------------------------------------------------------- utilidades */
  function h(tag, props = {}, ...children) {
    const node = document.createElement(tag);
    for (const [k, v] of Object.entries(props || {})) {
      if (v == null || v === false) continue;
      if (k === "class") node.className = v;
      else if (k === "style") node.setAttribute("style", v);
```

```
      else if (k.startsWith("on")) node.addEventListener(k.slice(2), v);
      else node.setAttribute(k, v === true ? "" : v);
    }
    for (const c of children.flat(Infinity)) {
      if (c == null || c === false) continue;
      node.append(c.nodeType ? c : document.createTextNode(String(c)));
    }
    return node;
  }
  const svg = (tag, attrs = {}, ...kids) => {
    const n = document.createElementNS("http://www.w3.org/2000/svg", tag);
    for (const [k, v] of Object.entries(attrs)) n.setAttribute(k, v);
    kids.forEach((c) => n.append(c));
    return n;
  };
  const fmtId = (id) => "#" + String(id).padStart(4, "0");
  const color = (type) => TYPE_COLORS[type] || "#667085";
  const key = (it) => `${it.id}|${it.form}`;

  function readableOn(hex) {
    const n = parseInt(hex.slice(1), 16);
    const [r, g, b] = [(n >> 16) & 255, (n >> 8) & 255, n & 255];
    return 0.299 * r + 0.587 * g + 0.114 * b > 150 ? "#1b2a41" : "#ffffff";
  }
  const chip = (type) =>
    h("span", { class: "chip", style: `background:${color(type)};color:${readableOn(color(type))}`
}, type);
  const chips = (it) =>
    h("span", { class: "chips" }, chip(it.type1), it.type2 ? chip(it.type2) : null);

  function showBanner(msg) { ui.banner.textContent = msg; ui.banner.hidden = false; }
  function hideBanner() { ui.banner.hidden = true; }
  function setStatus(kind, text) {
    ui.apiStatus.className = "status " + (kind ? "is-" + kind : "");
    ui.apiStatusText.textContent = text;
  }

  async function api(path, params = {}) {
    const qs = new URLSearchParams(Object.entries(params).filter(([, v]) => v !== "" && v != null));
    const url = `${API}${path}${qs.toString() ? "?" + qs : ""}`;
    let res;
    try {
      res = await fetch(url, { headers: { Accept: "application/json" } });
    } catch (err) {
      throw new Error("No se pudo conectar con la API. Revisa tu conexión y la URL configurada en
config.js.");
    }
    let body = null;
    try { body = await res.json(); } catch (_) { /* respuesta vacía */ }
    if (!res.ok) {
      const e = new Error((body && body.error) || `La API respondió con el estado ${res.status}.`);
      e.status = res.status;
      throw e;
    }
    return body;
  }

  /* ---------------------------------------------------------- tabla de datos */
  function renderRows() {
    ui.rows.replaceChildren();
    for (const it of state.items) {
      const tr = h("tr", {
        tabindex: "0", "data-key": key(it), style: `--type:${color(it.type1)}`,
        "aria-selected": state.selected && key(state.selected) === key(it) ? "true" : "false",
        class: state.selected && key(state.selected) === key(it) ? "is-selected" : "",
        onclick: () => select(it),
        onkeydown: (e) => { if (e.key === "Enter" || e.key === " ") { e.preventDefault();
select(it); } },
      },
        h("td", { class: "num" }, fmtId(it.id)),
        h("td", {}, h("span", { class: "cell-name" }, it.name),
          it.form !== "Base" ? h("span", { class: "cell-form" }, it.form) : null),
```

```
        h("td", {}, chips(it)),
        h("td", { class: "num" }, it.total),
        h("td", { class: "num" }, it.hp),
        h("td", { class: "num" }, it.attack),
        h("td", { class: "num" }, it.defense),
        h("td", { class: "num" }, it.speed),
        h("td", { class: "num" }, it.generation),
      );
      ui.rows.append(tr);
    }
    ui.more.hidden = !state.next;
  }

  function markSelected() {
    const k = state.selected ? key(state.selected) : null;
    for (const tr of ui.rows.children) {
      const on = tr.dataset.key === k;
      tr.classList.toggle("is-selected", on);
      tr.setAttribute("aria-selected", on ? "true" : "false");
    }
  }

  function showEmpty(msg) {
    ui.empty.textContent = msg;
    ui.empty.hidden = !msg;
    document.getElementById("grid").hidden = !!msg;
  }

  /* ------------------------------------------------------------------ listado */
  function filterText() {
    const parts = [];
    if (ui.type.value) parts.push(`tipo ${ui.type.value}`);
    if (ui.gen.value) parts.push(`generación ${ui.gen.value}`);
    return parts.length ? ` · ${parts.join(", ")}` : "";
  }

  async function loadList({ append = false, name = "" } = {}) {
    if (state.busy) return;
    state.busy = true;
    ui.more.disabled = true;
    hideBanner();
    try {
      const data = await api("/pokemon", {
        limit: PAGE_SIZE, type: ui.type.value, generation: ui.gen.value,
        name, cursor: append ? state.next : "",
      });
      state.items = append ? state.items.concat(data.items) : data.items;
      state.next = data.next_cursor;
      showEmpty(state.items.length ? "" :
        name ? `No hay registros cuyo nombre contenga “${name}”. Revisa la ortografía o escribe
menos letras.`
             : "No hay registros con esos filtros.");
      if (name) {
        const total = data.total_matches ?? data.count;
        ui.summary.textContent = `${total} coincidencia${total === 1 ? "" : "s"} con
“${name}”${filterText()}` +
          (total > data.count ? ` · se muestran las primeras ${data.count}` : "");
      } else {
        ui.summary.textContent = `${state.items.length} registros cargados${filterText()}` +
          (state.next ? " · hay más" : " · fin de la lista");
      }
      renderRows();
      if (!append && state.items.length && !state.selected) select(state.items[0]);
    } catch (err) {
      showBanner(err.message);
      ui.summary.textContent = "";
    } finally {
      state.busy = false;
      ui.more.disabled = false;
    }
  }
```

```
  async function searchById(raw) {
    if (!/^\d{1,4}$/.test(raw)) {
      ui.summary.textContent = "El ID debe ser un número entero, por ejemplo 25.";
      return;
    }
    hideBanner();
    try {
      const data = await api(`/pokemon/${Number(raw)}`);
      state.formsCache.set(data.id, data.forms);
      state.items = data.forms;
      state.next = null;
      showEmpty("");
      ui.summary.textContent = `ID ${fmtId(data.id)} · ${data.forms.length} forma${data.forms.length
=== 1 ? "" : "s"} registrada${data.forms.length === 1 ? "" : "s"}`;
      renderRows();
      select(data.forms[0]);
    } catch (err) {
      if (err.status === 404) {
        state.items = []; state.next = null;
        renderRows();
        showEmpty(`No existe un registro con el ID ${raw}. En este dataset los IDs van de 1 a
1025.`);
        ui.summary.textContent = "";
      } else showBanner(err.message);
    }
  }

  /* ---------------------------------------------------------------- ficha */
  async function select(item) {
    state.selected = item;
    markSelected();
    renderEntry(item, state.formsCache.get(item.id) || [item]);
    if (!state.formsCache.has(item.id)) {
      try {
        const data = await api(`/pokemon/${item.id}`);
        state.formsCache.set(item.id, data.forms);
        if (state.selected && state.selected.id === item.id) renderEntry(state.selected,
data.forms);
      } catch (_) { /* la ficha ya se pintó con los datos de la fila */ }
    }
  }

  function radar(item) {
    const cx = 150, cy = 125, R = 82, n = STATS.length;
    const point = (i, r) => {
      const a = -Math.PI / 2 + (2 * Math.PI * i) / n;
      return [cx + r * Math.cos(a), cy + r * Math.sin(a)];
    };
    const root = svg("svg", { class: "radar", viewBox: "0 0 300 250", role: "img",
      "aria-label": "Gráfica radial de estadísticas: " + STATS.map(([k, l]) => `${l}
${item[k]}`).join(", ") });
    [0.25, 0.5, 0.75, 1].forEach((f) =>
      root.append(svg("polygon", { class: "ring", points: STATS.map((_, i) => point(i, R *
f).join(",")).join(" ") })));
    STATS.forEach((_, i) => {
      const [x, y] = point(i, R);
      root.append(svg("line", { class: "axis", x1: cx, y1: cy, x2: x, y2: y }));
    });
    root.append(svg("polygon", { class: "shape",
      points: STATS.map(([k], i) => point(i, R * Math.min(item[k] / STAT_MAX, 1)).join(",")).join("
") }));
    STATS.forEach(([, label], i) => {
      const [x, y] = point(i, R + 18);
      const t = svg("text", { x, y: y + 4, "text-anchor": x < cx - 5 ? "end" : x > cx + 5 ? "start"
: "middle" });
      t.textContent = label;
      root.append(t);
    });
    return root;
  }

  function renderEntry(item, forms) {
```

```
    ui.entry.style.setProperty("--type", color(item.type1));
    // replaceChildren recibe nodos o textos; por eso se aplana el arreglo y se descartan los null.
    ui.entry.replaceChildren(...[
      h("p", { class: "big-num", "aria-hidden": "true" }, fmtId(item.id)),
      h("h2", {}, item.name),
      h("div", { class: "entry__meta" }, chips(item),
        h("span", {}, `Generación ${item.generation}`),
        item.form !== "Base" ? h("span", {}, `· ${item.form}`) : null),
      forms.length > 1 ? h("div", { class: "forms", role: "group", "aria-label": "Formas de esta
especie" },
        forms.map((f) => h("button", {
          type: "button", "aria-pressed": key(f) === key(item) ? "true" : "false",
          onclick: () => select(f),
        }, f.form === "Base" ? "Base" : f.form))) : null,
      STATS.map(([k, label]) =>
        h("div", { class: "stat" },
          h("span", {}, label),
          h("span", { class: "stat__bar", "aria-hidden": "true" },
            h("span", { class: "stat__fill", style: `width:${Math.min(100, (item[k] / STAT_MAX) *
100)}%` })),
          h("span", { class: "stat__val" }, item[k]))),
      h("div", { class: "stat stat--total" }, h("span", {}, "Total"), h("span", { class: "stat__val"
}, item.total)),
      radar(item),
    ].flat(Infinity).filter(Boolean));
  }

  /* ---------------------------------------------------------------- panorama */
  async function loadOverview() {
    if (state.overviewLoaded) return;
    ui.overview.replaceChildren(h("p", { class: "empty" }, "Calculando estadísticas…"));
    try {
      const s = await api("/stats");
      state.overviewLoaded = true;
      const maxType = Math.max(...s.by_type.map((t) => t.count));
      const maxGen = Math.max(...s.by_generation.map((g) => g.count));
      ui.overview.replaceChildren(
        card("third", "Registros en la base de datos", kpi(s.total_records.toLocaleString("es-CO"),
"filas cargadas desde el CSV")),
        card("third", "Especies distintas", kpi(s.unique_species.toLocaleString("es-CO"), "IDs
únicos de la Pokédex")),
        card("third", "Total promedio", kpi(s.average_total, "suma media de las seis
estadísticas")),
        card("half", "Registros por tipo primario",
          s.by_type.map((t) => h("div", { class: "hbar" },
            h("span", {}, t.type),
            h("span", { class: "hbar__track" }, h("span", { class: "hbar__fill", style: `--
c:${color(t.type)};width:${(t.count / maxType) * 100}%` })),
            h("span", { class: "hbar__val" }, t.count)))),
        card("half", "Registros por generación",
          h("div", { class: "cols", role: "img", "aria-label": "Registros por generación: " +
s.by_generation.map((g) => `generación ${g.generation}, ${g.count}`).join("; ") },
            s.by_generation.map((g) => h("div", { class: "cols__item" },
              h("span", { class: "cols__n" }, g.count),
              h("span", { class: "cols__bar", style: `height:${(g.count / maxGen) * 80}%` }),
              h("span", {}, "G" + g.generation))))),
        card("half", "Promedio de cada estadística",
          STATS.map(([k, label]) => h("div", { class: "hbar" },
            h("span", {}, label),
            h("span", { class: "hbar__track" }, h("span", { class: "hbar__fill", style:
`width:${(s.average_stats[k] / 130) * 100}%` })),
            h("span", { class: "hbar__val" }, s.average_stats[k])))),
        card("half", "Los cinco con mayor total",
          h("ol", { class: "top5" }, s.top_total.map((t) => h("li", {},
            h("button", { type: "button", onclick: () => jumpToId(t.id) },
              `${fmtId(t.id)} ${t.name}${t.form !== "Base" ? " (" + t.form + ")" : ""}`),
            h("b", {}, t.total))))),
      );
    } catch (err) {
      ui.overview.replaceChildren(h("p", { class: "empty" }, err.message));
    }
  }
  const card = (size, title, ...body) =>
```

```
    h("section", { class: `card card--${size}` }, h("h2", {}, title), body);
  const kpi = (value, label) => h("dl", { class: "kpi" }, h("dd", {}, value), h("dt", {}, label));

  function jumpToId(id) {
    switchView("explore");
    setMode("id");
    ui.q.value = String(id);
    searchById(String(id));
    if (ui.entry.scrollIntoView) ui.entry.scrollIntoView({ behavior: "smooth", block: "nearest" });
  }

  /* ---------------------------------------------------------- interacción */
  function setMode(mode) {
    state.mode = mode;
    document.querySelector(`input[name=mode][value=${mode}]`).checked = true;
    ui.q.value = "";
    if (mode === "id") {
      ui.q.inputMode = "numeric"; ui.q.placeholder = "25"; ui.q.maxLength = 4;
      ui.qLabel.textContent = "Número de la Pokédex";
    } else {
      ui.q.inputMode = "text"; ui.q.placeholder = "pika"; ui.q.maxLength = 40;
      ui.qLabel.textContent = "Nombre completo o parcial";
    }
  }

  function switchView(view) {
    state.view = view;
    for (const v of ["explore", "overview"]) {
      const on = v === view;
      $("view-" + v).hidden = !on;
      $("tab-" + v).classList.toggle("is-active", on);
      $("tab-" + v).setAttribute("aria-selected", on ? "true" : "false");
    }
    if (view === "overview") loadOverview();
  }

  function onFilterChange() {
    state.filtersTouched = true; // el usuario eligió filtros: se respetan en las búsquedas
    loadList({ name: state.mode === "name" ? ui.q.value.trim() : "" });
  }

  function init() {
    Object.keys(TYPE_COLORS).sort().forEach((t) => ui.type.append(h("option", { value: t }, t)));
    for (let g = 1; g <= 9; g++) ui.gen.append(h("option", { value: g }, `Generación ${g}`));
    ui.gen.value = "1";

    document.querySelectorAll("input[name=mode]").forEach((r) =>
      r.addEventListener("change", () => setMode(r.value)));
    ui.form.addEventListener("submit", (e) => {
      e.preventDefault();
      const value = ui.q.value.trim();
      if (state.mode === "id") searchById(value);
      else {
        // La generación 1 es solo el filtro inicial: al buscar por nombre se amplía a todas
        // las generaciones, salvo que la persona haya elegido filtros por su cuenta.
        if (!state.filtersTouched) ui.gen.value = "";
        loadList({ name: value });
      }
    });
    ui.clear.addEventListener("click", () => { ui.q.value = ""; state.items = []; loadList();
ui.q.focus(); });
    ui.type.addEventListener("change", onFilterChange);
    ui.gen.addEventListener("change", onFilterChange);
    ui.more.addEventListener("click", () => loadList({ append: true, name: "" }));
    $("tab-explore").addEventListener("click", () => switchView("explore"));
    $("tab-overview").addEventListener("click", () => switchView("overview"));

    const t0 = performance.now();
    api("/health")
      .then(() => setStatus("ok", `API en línea · ${Math.round(performance.now() - t0)} ms`))
      .catch((err) => { setStatus("bad", "API sin conexión"); showBanner(err.message); });
```

```
    loadList();
  }

  init();
})();
```

13. AGRADECIMIENTO

Agradezco al profesor, Ing. Víctor Augusto Pedraza Bonilla, por el acompañamiento y la orientación a lo largo del módulo de Contenerización de Aplicaciones de Software. Sus lecturas, sus explicaciones y la claridad con que planteó cada actividad permitieron entender la contenerización no solo como una técnica, sino como una decisión de arquitectura, y comparar con criterio las distintas formas de desplegar una aplicación: desde los contenedores en la Unidad 3, con la orquestación de Kubernetes, hasta el enfoque serverless de esta actividad, donde la infraestructura deja de ser una preocupación del equipo.

Agradezco también a los compañeros del aula de la Maestría en Arquitectura de Software, por la riqueza del conocimiento académico y profesional que se comparte en las aulas, y que hizo posible contrastar distintas formas de desplegar aplicaciones en la nube. Las discusiones, las preguntas y las experiencias de cada uno ayudaron a mirar los problemas desde otros ángulos y a justificar mejor las decisiones de diseño.

Finalmente, agradezco al Politécnico Grancolombiano por ofrecer un espacio de formación en el que la teoría se lleva a la práctica con actividades reales, como esta, en las que cada concepto se prueba con una aplicación que funciona y se puede ver publicada.